



UNIVERSITÀ DI PISA

DIPARTIMENTO DI MATEMATICA

Laurea Triennale in Matematica

Canonical Polyadic decomposition and applications

Relatore:

Prof. Leonardo Robol

Candidato:

Alberto Defendi

ANNO ACCADEMICO 2025/2026

SOMMARIO

La decomposizione CP (Canonical Polyadic Decomposition) di rango r di un tensore è la sua scomposizione nella somma di r tensori di rango uno, una generalizzazione naturale della decomposizione di matrici. Oltre all'importanza teorica nel calcolo tensoriale, è uno strumento utile in molte aree scientifiche: per citarne alcune, spettroscopia, signal processing, neurologia, e analisi dei dati. In molte di queste applicazioni l'unicità è una proprietà fondamentale per rappresentare fedelmente le componenti di sistemi complessi—si pensi al problema di modellare i neuroni di un sistema nervoso tramite un tensore per studiarne gli impulsi, anche detto *blind source separation*.

Il primo obiettivo di questo elaborato è dimostrare che, sotto opportune ipotesi, la fattorizzazione CP è unica. Questo risultato, originariamente dimostrato da Kruskal nel 1970 attraverso il concetto di Kruskal rank, è stato riscritto di recente in una dimostrazione geometrica. Inoltre, osserveremo alcune peculiarità teoriche che distinguono i tensori dalle matrici, ponendo le basi per l'utilizzo del calcolo tensoriale nella teoria della complessità.

Successivamente, applicheremo questi strumenti al problema della complessità della moltiplicazione tra matrici. Essendo una mappa bilineare, l'algoritmo di moltiplicazione può essere codificato nel suo tensore strutturale a tre indici, e poi messo in corrispondenza con un algoritmo per valutarlo. Introduciamo la nozione di complessità aritmetica del prodotto tra matrici per definire l'esponente asintotico ω che ne misura la complessità ottimale, e sfrutteremo il Teorema di Strassen per mostrare la corrispondenza tra la ricerca di algoritmi veloci e il calcolo del rango CP del tensore. Come conseguenza, ridurre lo studio della complessità ai tensori, anziché all'analisi diretta delle equazioni algebriche, semplifica notevolmente la ricerca di algoritmi più efficienti; una sfida aperta da decenni, dall'algoritmo di Strassen (del 1970), fino alle recenti scoperte guidate dall'intelligenza artificiale.

Infine, estenderemo l'analisi dagli algoritmi di moltiplicazione esatti agli algoritmi approssimati (APA), introdotti da Bini et al. In questo contesto emerge il fenomeno topologico del border rank, per il quale una successione di tensori di rango inferiore può convergere a un tensore di rango superiore. Ciò permette, in termini di algoritmi, approssimare un algoritmo con una successione di algoritmi con un numero di moltiplicazione più basso. Da cui potremo sostituire il rango tensoriale con il border rank nel calcolo di ω , permettendo di ottenere stime asintotiche più stringenti, al prezzo di un errore algoritmico arbitrariamente piccolo.

INDICE

1	LA FATTORIZZAZIONE CP	1
1.1	Tensori	1
1.2	Il prodotto esterno	2
1.3	La fattorizzazione CP	6
1.4	Verso l'unicità	9
1.5	Il problema del rango	12
1.6	Il teorema del rango	14
1.7	Rango tipico	16
1.8	Operazioni nel formato CP	16
1.9	Calcolo della fattorizzazione CP	16
2	APPLICAZIONI DELLA CP ALLA COMPLESSITÀ ARITMETICA	19
2.1	Mappe multilineari e moltiplicazione tra matrici	19
2.2	L'esponente della moltiplicazione tra matrici	21
2.3	Decomposizione low-rank di forme bilineari	23
2.4	Rappresentazione del prodotto tra matrici	25
2.5	Alcune stime sull'esponente	26
2.6	Il rango come misura della complessità	27
2.7	Algoritmi di moltiplicazione veloce	31
2.8	Algoritmi APA	34
2.9	Errore algoritmico	37
3	APPLICAZIONI DELLA CP	39
3.1	Fluorescenza spectroscopica	39
3.2	Cocktail party problem	41
4	APPENDICE	43
4.1	Il master method	43
	ACRONIMI	45

Indice

GLOSSARIO 47

BIBLIOGRAFIA 49

1 LA FATTORIZZAZIONE CP

1.1 TENSORI

Denotiamo gli spazi vettoriali con le lettere A, B, C, V, W e A_j , e le dimensioni con le lettere $\mathbf{a}, \mathbf{b}, \mathbf{c}$, etc. Se $v_1, \dots, v_p \in V$, $\langle v_1, \dots, v_p \rangle \subseteq V$ denota lo span lineare di $v_1, \dots, v_p$. Indichiamo con $V^* = \{f : V \rightarrow \mathbb{K} \mid f \text{ lineare}\}$ lo *spazio duale* dello spazio V , e per ogni $\alpha \in V^*$, denotiamo con $\alpha^\perp = \{v \in V \mid \alpha(v) = 0\}$ l'*annullatore* di α . Similmente, dato un sottospazio vettoriale $U \subseteq V$, definiamo l'annullatore di U come $U^\perp = \{\alpha \in V^* \mid \alpha(u) = 0, \forall u \in U\}$.

Definizione 1.1.1. Denotiamo l'insieme dei numeri naturali da 1 a $n \in \mathbb{N}$ come $\underline{n} = \{1, \dots, n\}$, e il prodotto cartesiano tra $\underline{n}$ e $\underline{m}$ come $\underline{n} \times \underline{m} = \{(i, j) : i \in \underline{n}, j \in \underline{m}\}$.

Osservazione 1.1.2. Un tensore $T \in \mathbb{R}^{m \times n \times 1}$ è una matrice $m \times n$, e più in generale un tensore $T \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$ è un tensore di dimensione $n_1 \times n_2 \times \dots \times n_d$.

Definizione 1.1.3. Un *tensore* T di ordine d è un array multidimensionale con entrate definite sul campo $\mathbb{K}$, che denotiamo come $T \in \mathbb{K}^{n_1 \times n_2 \times \dots \times n_d}$. Indicizziamo gli elementi di un tensore come $T_{i_1, i_2, \dots, i_d}$, dove $(i_1, i_2, \dots, i_d) \in [n_1] \times [n_2] \times \dots \times [n_d]$.

Osservazione 1.1.4. Per array intendiamo un arrangiamento di elementi di $\mathbb{K}$ secondo gli indici $(i_1, i_2, \dots, i_d) \in [n_1] \times [n_2] \times \dots \times [n_d]$. Trattare la buona definizione dei tensori dal punto di vista algebrico esula dallo scopo del presente lavoro, tuttavia il lettore interessato a una visione algebrica più ampia, può guardare.

Esempio 1.1.5. (Tensori che rappresentano applicazioni multilineari)

Osservazione 1.1.6. L'insieme dei tensori $\mathbb{R}^{n_1 \times \dots \times n_d}$ forma un $\mathbb{R}$ -spazio vettoriale, dotato del prodotto scalare euclideo, con le operazioni

- Addizione: $(T_1 + T_2)(i_1, \dots, i_d) = T_1(i_1, \dots, i_d) + T_2(i_1, \dots, i_d)$.
- Moltiplicazione per scalare: Per ogni $\alpha \in \mathbb{R}$, $\alpha(T(i_1, \dots, i_d)) = (\alpha T)(i_1, \dots, i_d)$

Usare emph quando si introduce un termine nuovo, inserire tabella notazione all'inizio

Vorrei introdurre il prima possibile la CP

trovare notazione alternativa per $[n]$ perché si confonde con il proiettivo

Cita landberg o Hackbush)

Scrivere

ADD: Esempi di tensori. Tra cui il tensore che indica (quello del prodotto di matrici)

- Prodotto scalare: $\langle T_1, T_2 \rangle = \text{vec}(T_1)^\top \text{vec}(T_2)$
- Norma: $\|\text{vec}(T)\|$

1.2 IL PRODOTTO ESTERNO

Dati due vettori $u = [u_1, \dots, u_m]^\top \in \mathbb{R}^m$ e $v = [v_1, \dots, v_n]^\top \in \mathbb{R}^n$, possiamo introdurre un'operazione $\circ$ tra u e v definita come

$$u \circ v = uv^\top = \begin{bmatrix} u_1 v_1 & u_1 v_2 & \dots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \dots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \dots & u_m v_n \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

Possiamo generalizzare tale costruzione a più dimensioni:

Definizione 1.2.1 (Prodotto esterno). Dati dei vettori $v^1 \in \mathbb{R}^{n_1}, \dots, v^d \in \mathbb{R}^{n_d}$, loro *prodotto esterno*, anche detto *prodotto tensore*, è il tensore

$$(1.1) \quad T = v^1 \circ v^2 \circ \dots \circ v^d \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$$

tale che la sua scrittura in componenti è

$$T_{i_1, i_2, \dots, i_d} = v_{i_1}^1 v_{i_2}^2 \dots v_{i_d}^d = \prod_{j=1}^d v_{i_j}^j.$$

Definizione 1.2.2 (Tensore di rango uno). Un tensore $T \in \mathbb{R}^{n_1 \times \dots \times n_d}$ ha *rango uno* se si può scrivere come nell'Equazione 1.1.

Questo ci permette di definire la seguente quantità fondamentale:

Definizione 1.2.3 (Rango di un tensore). Il rango di un tensore $T \in \mathbb{R}^{n_1 \times \dots \times n_d}$ è il minimo intero positivo r tale che T si può scrivere come somma di tensori di rango uno, e scriviamo $\mathbf{R}(T) = r$.

Introduciamo alcuni ingredienti utili a definire la fattorizzazione CP.

Definizione 1.2.4 (Prodotto di Kroneker). Date due matrici $A \in \mathbb{R}^{m \times n}$ e $B \in \mathbb{R}^{p \times q}$, il loro prodotto di Kroneker è la matrice

$$C = A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B & \dots & a_{1n}B \\ a_{21}B & a_{22}B & \dots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \dots & a_{mn}B \end{bmatrix} \in \mathbb{R}^{mp \times nq}, \text{ dove } c_{kl} = a_{ij}b_{nl},$$

dove $\alpha = \mathbb{L}^*(i, k) = \mathbb{L}(k, i)$ e $\beta = \mathbb{L}^*(j, l) = \mathbb{L}(l, j)$

riscrivere con $\mathbb{L}^*$. Vedi definizione nelle note

Osservazione 1.2.5 (Collegamento con il prodotto tensore). Dati due vettori $u \in \mathbb{R}^m$ e $v \in \mathbb{R}^n$, il loro prodotto di Kroneker si può vedere come vettorizzazione del prodotto vettore, ossia

(1.2)

$$\text{vec}(v \otimes u) = \text{vec} \left(\begin{bmatrix} v_1 u_1 & v_1 u_2 & \dots & v_1 u_m \\ v_2 u_1 & v_2 u_2 & \dots & v_2 u_m \\ \vdots & \vdots & \ddots & \vdots \\ v_n u_1 & v_n u_2 & \dots & v_n u_m \end{bmatrix} \right) = [u_1 v \mid u_2 v \mid \dots \mid u_m v] = u \otimes v$$

Notiamo che tale operazione scambia l'ordine tra i vettori.

Breve motivazione dell'ordinamento naturale

Definizione 1.2.6 (Ordinamento naturale). L'ordinamento naturale (o natural ordering) sull'insieme $[n_1] \times \dots \times [n_d]$ è la mappa biunivoca

$$\mathbb{L}: [n_1] \times \dots \times [n_d] \rightarrow \left[\prod_{j=1}^d n_j \right]$$

$$(i_1, i_2, \dots, i_d) \mapsto 1 + s_1(i_1 - 1) + \dots + s_d(i_d - 1) = 1 + \sum_{j=1}^d s_j(i_j - 1),$$

dove i coefficienti s_j sono chiamati *passi j-esimi* e sono definiti come $s_1 = 1$ e $s_{j+1} = n_1 \cdots n_j = \prod_{h=1}^j n_h = s_j n_{j+1}$, per ogni $j \in [d-1]$. L'inversa di $\mathbb{L}$ è data dalla mappa

$$\mathbb{L}^{-1}: \left[\prod_{j=1}^d n_j \right] \rightarrow [n_1] \times \dots \times [n_d]$$

$$l \mapsto \left(1 + \left\lfloor \frac{(l-1) \bmod (n_1 s_1)}{s_1} \right\rfloor, \dots, 1 + \left\lfloor \frac{(l-1) \bmod (n_d s_d)}{s_d} \right\rfloor \right).$$

Le mappe $\mathbb{L}$ e $\mathbb{L}^{-1}$ sono l'una l'inversa dell'altra, e sono anche dette linearizzazione e tensorizzazione degli indici, rispettivamente, in quanto permettono di passare da un indice tensoriale a uno lineare, e viceversa.

Buona definizione. Segue direttamente dalla definizione di $\mathbb{L}$ e dalle proprietà del modulo, infatti,

$$\begin{aligned}\mathbb{L}^{-1}(\mathbb{L}(i_1, i_2, \dots, i_d)) &= \mathbb{L}^{-1}(1 + s_1(i_1 - 1) + \dots + s_d(i_d - 1)) \\ &= \left(1 + \left\lfloor \frac{\sum_{j=1}^d s_j(i_j - 1) \bmod (n_1 s_1)}{s_1} \right\rfloor, \dots, 1 + \left\lfloor \frac{\sum_{j=1}^d s_j(i_j - 1) \bmod (n_d s_d)}{s_d} \right\rfloor\right) \\ &= \left(1 + \left\lfloor \sum_{j=1}^d \frac{s_j}{s_1} (i_j - 1) \bmod (n_1 s_1) \right\rfloor, \dots, 1 + \left\lfloor \sum_{j=1}^d \frac{s_j}{s_d} (i_j - 1) \bmod (n_d s_d) \right\rfloor\right) \\ &= (i_1, i_2, \dots, i_d),\end{aligned}$$

dove nell'ultimo passaggio abbiamo usato che $s_j = \prod_{k=1}^d n_k$ e il fatto che $\lfloor x \rfloor = 0$ per ogni $x \in [0, 1)$. Viceversa,

$$\begin{aligned}\mathbb{L}(\mathbb{L}^{-1}(l)) &= \mathbb{L}\left(1 + \left\lfloor \frac{(l-1) \bmod (n_1 s_1)}{s_1} \right\rfloor, \dots, 1 + \left\lfloor \frac{(l-1) \bmod (n_d s_d)}{s_d} \right\rfloor\right) = \\ &= 1 + s_1 \left(\left\lfloor \frac{(l-1) \bmod (n_1 s_1)}{s_1} \right\rfloor \right) + \dots + s_d \left(\left\lfloor \frac{(l-1) \bmod (n_d s_d)}{s_d} \right\rfloor \right)\end{aligned}$$

FINIRE

□

Esempio 1.2.7. È utile visualizzare il caso 3-dimensionale: fissato un tensore $T \in \mathbb{R}^{m \times n \times p}$ indicizzato con (i, j, k) , abbiamo che $s_1 = 1, s_2 = m$ e $s_3 = mn$, dunque

$$\mathbb{L}(i, j, k) = 1 + (i - 1) + m(j - 1) + mn(k - 1) = i + m(j - 1) + mn(k - 1),$$

mentre l'inversa è

$$\mathbb{L}^{-1}(i, j, k) = \left(\lfloor (i-1) \bmod (m) \rfloor, 1 + \left\lfloor \frac{(i-1) \bmod (mn)}{m} \right\rfloor, 1 + \left\lfloor \frac{(i-1) \bmod (mnp)}{mn} \right\rfloor \right).$$

Definizione 1.2.8 (Reverse ordering). *L'ordinamento inverso* sull'insieme $[n_1] \times \cdots \times [n_d]$ è la mappa biunivoca

$$\begin{aligned} \mathbb{L}^* : [n_1] \times \cdots \times [n_d] &\rightarrow \left[\prod_{j=1}^d n_j \right] \\ (i_1, i_2, \dots, i_d) &\mapsto 1 + \sum_{j=1}^d s_j^* (i_j - 1), \end{aligned}$$

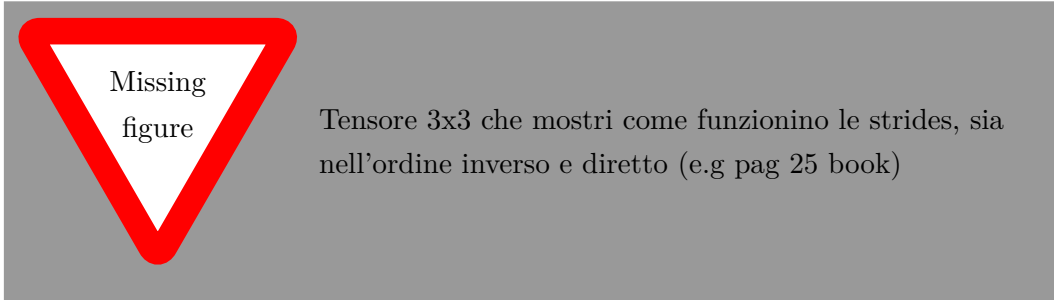
dove $s_d^* = 1$ e $s_{j-1}^* = \prod_{h=j}^d n_h = s_j^* n_j$, per $j = d, \dots, 2$. La sua inversa è la mappa

$$\begin{aligned} (\mathbb{L}^*)^{-1} : \left[\prod_{j=1}^d n_j \right] &\rightarrow [n_1] \times \cdots \times [n_d] \\ l &\mapsto \left(1 + \left\lfloor \frac{(l-1) \bmod (n_1 s_1^*)}{s_1^*} \right\rfloor, \dots, 1 + \left\lfloor \frac{(l-1) \bmod (n_d s_d^*)}{s_d^*} \right\rfloor \right) \end{aligned}$$

.

Esempio 1.2.9. Nel caso di un tensore $T \in \mathbb{R}^{m \times n \times p}$, i passi sono $s_3^* = 1$, $s_2^* = p$ e $s_1^* = np$.

Osservazione 1.2.10. I passi determinano l'ordine di percorrenza del tensore nelle definizioni di $\mathbb{L}$ e $\mathbb{L}^*$.

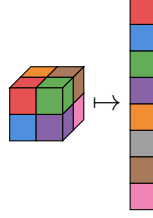


Analogamente alle matrici, i tensori vengono salvati in memoria come vettori, risulta quindi naturale definire la seguente operazione

Definizione 1.2.11. Dato un tensore T , possiamo definire l'operazione che trasforma un tensore in vettore, tramite la mappa di vettorizzazione

$$\text{vec}: \mathbb{R}^{n_1 \times n_2 \times \cdots \times n_d} \rightarrow \mathbb{R}^{\prod_{j=1}^d n_j}$$

$$T \mapsto x$$



dove $x_l = T(\mathbb{L}^{-1}(l; n_1, n_2, \dots, n_d))$.

Osservazione 1.2.12. L'operatore di trasposizione per matrici induce l'ordinamento inverso sulla vettorizzazione. Consideriamo ad esempio il caso 2×2 :

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad A^\top = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{bmatrix} \implies \text{vec}(A) = \begin{bmatrix} a_{11} \\ a_{21} \\ a_{12} \\ a_{22} \end{bmatrix}, \quad \text{vec}(A^\top) = \begin{bmatrix} a_{11} \\ a_{12} \\ a_{21} \\ a_{22} \end{bmatrix}.$$

Nel caso matriciale si usa riferirsi all'ordine naturale come *column-major* e a quello inverso come *row-major*, specialmente nel contesto dei linguaggi di programmazione.

Check

1.3 LA FATTORIZZAZIONE CP

Introduciamo una scrittura di un tensore in tensori elementari, utile a risolvere i seguenti problemi che affronteremo nei prossimi capitoli.

Il problema inverso del rango: Dato un intero positivo r , nel caso matriciale è, in linea teorica, un problema semplice costruire una matrice di rango r ; mentre dal punto di vista computazionale, esistono algoritmi polinomiali per approssimare il rango. Nel caso dei tensori, il problema è difficile su entrambi i fronti. Nella teoria, trovare un tensore di un certo rango è come trovare un ago in un pagliaio; nella pratica, approssimare il tensore, o il rango, è numericamente difficile. Questo complica la risoluzione di molti problemi che possono essere modellati tramite tensori, come vedremo ad esempio nel Capitolo 2.

Risparmio computazionale: Cerchiamo una decomposizione di rango basso di un tensore $T \in \mathbb{R}^{n_1 \times \dots \times n_d}$ per ridurre il costo di rappresentazione in memoria e delle operazioni algebriche che lo coinvolgono. Nel caso matriciale, data una matrice $X \in \mathbb{R}^{n_1 \times n_2}$ di rango r , possiamo scrivere

$$X = UV^\top = [u_1 | \dots | u_r] [v_1 | \dots | v_r]^\top = u_1 v_1^\top + \dots + u_r v_r^\top = u_1 \circ v_1 + \dots + u_r \circ v_r,$$

per certe matrici $U \in \mathbb{R}^{n_1 \times r}$ e $V \in \mathbb{R}^{n_2 \times r}$. Questo tipo di decomposizione è vantaggioso in termini di memoria: infatti

$$\text{vec}(X) = v_1 \otimes u_1 + \dots + v_r \otimes u_r,$$

quindi memorizzare U e V costa $O((n_1 + n_2)r)$ invece che $O(n_1 n_2)$. Vorremmo quindi riadattare questa decomposizione per memorizzare T con costo $O((n_1 + \dots + n_d)r)$ invece che $O(n_1 \dots n_d)$. Tuttavia, generalizzare questa decomposizione a più dimensioni ha numerosi ostacoli .

discorso unicità

ref to ostacoli

Definizione 1.3.1 (Canonical Polyadic Decomposition (CP)). Dati degli spazi vettoriali $A_1 \otimes \dots \otimes A_d$ e dei vettori $\{u_1^t, \dots, u_{n_t}^t\} \subset A_t$, diciamo che un tensore $T \in A_1 \otimes \dots \otimes A_d$ ammette una *decomposizione CP* di rango $r \in \mathbb{N}$ se si può scrivere come

$$T = u_1^1 \otimes u_1^2 \otimes \dots \otimes u_1^d + \dots + u_r^1 \otimes u_r^2 \otimes \dots \otimes u_r^d.$$

serve ipotesi sui vettori?

Osservazione 1.3.2. La definizione è invariante per cambiamento di base.

Osservazione 1.3.3 (Versione originale). In letteratura spesso la CP viene definita in coordinate $\mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$ tramite delle matrici $U_1, \dots, U_d$ tali che $U_t = [u_1^t | \dots | u_r^t] \in \mathbb{R}^{n_t \times r}$ tali per cui

$$T = u_1^1 \otimes u_1^2 \otimes \dots \otimes u_1^d + \dots + u_r^1 \otimes u_r^2 \otimes \dots \otimes u_r^d \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}.$$

In tal caso, scriveremo $T = \llbracket U_1, \dots, U_d \rrbracket$.

Osservazione 1.3.4. Fissando la notazione sopra, la fattorizzazione CP si può scrivere nelle seguenti formulazioni equivalenti:

- (i) $T = \sum_{j=1}^r u_j^1 \circ \dots \circ u_j^d.$
- (ii) $\text{vec}(T) = \sum_{j=1}^r u_j^d \otimes \dots \otimes u_j^1.$

la 3 non torna

$$(iii) \quad T_{i_1, i_2, \dots, i_d} = \sum_{j=1}^r U_1(i_1, j) U_2(i_2, j) \cdots U_d(i_d, j).$$

Esempio 1.3.5 (De Lathauwer [11]). Consideriamo il tensore T di taglia $2 \times 2 \times 2$ definito da

$$a_{111} = -a_{121} = 3, \quad a_{211} = -a_{221} = 6, \quad a_{112} = -a_{122} = 1, \quad a_{212} = -a_{222} = 2.$$

I vettori dei modi 1, 2 e 3 sono le colonne delle matrici

$$\mathbf{T}_{(1)} = \begin{bmatrix} 3 & -3 & 1 & -1 \\ 6 & -6 & 2 & -2 \end{bmatrix}, \quad \mathbf{T}_{(2)} = \begin{bmatrix} 3 & 6 & 1 & 2 \\ -3 & -6 & -1 & -2 \end{bmatrix}, \quad \mathbf{T}_{(3)} = \begin{bmatrix} 3 & 6 & -3 & -6 \\ 1 & 2 & -1 & -2 \end{bmatrix},$$

rispettivamente. Il tensore ha rango uno perché si può decomporre come

$$T = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \circ \begin{bmatrix} 1 \\ -1 \end{bmatrix} \circ \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

Pensando la terza dimensione nel verso entrante al foglio, possiamo riassumere i passaggi svolti per fattorizzare T , dove è stata applicata più volte la definizione di prodotto tensore:

$$\begin{bmatrix} 1 & -1 \\ 2 & 3 \\ 6 & -6 \end{bmatrix}^2 = {}_3 \left(\begin{bmatrix} 1 \\ 2 \end{bmatrix} \circ \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \circ \begin{bmatrix} 1 \\ -1 \end{bmatrix} \circ \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

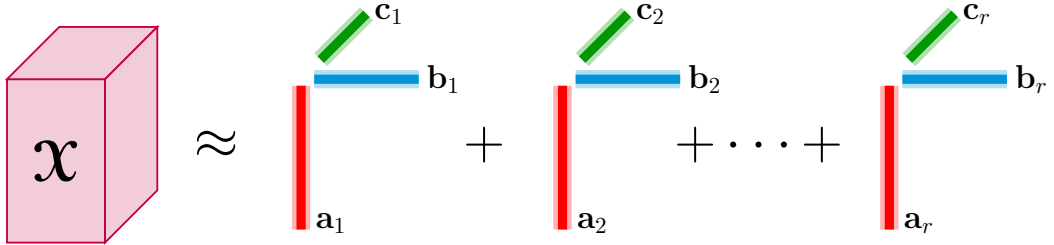


Figura 1.1: Fattorizzazione CP di un tensore $T = \llbracket A, B, C \rrbracket \in \mathbb{R}^{n \times m \times p}$, dove i vettori a_l, b_l, c_l indicano le colonne l -esime di A, B, C .

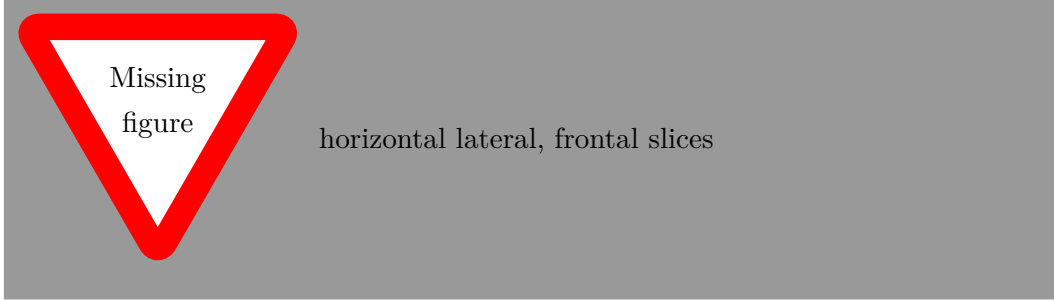
Osservazione 1.3.6. Il rango CP di T coincide per definizione al rango di T .

Osservazione 1.3.7 (Risparmio in memoria della CP). La fattorizzazione CP di un tensore T di rango r permette di ridurre notevolmente il costo di memorizzazione del tensore, in quanto richiede $O((n_1 + \dots + n_d)r) \ll O(\prod_{i=1}^d n_i)$.

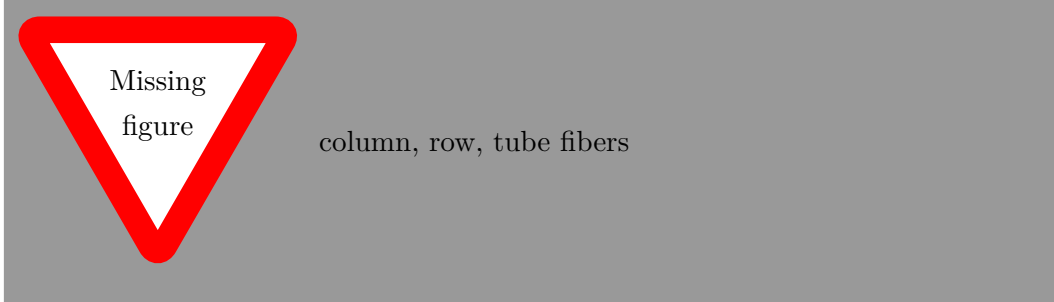
1.4 VERSO L'UNICITÀ

Cambiare

Definizione 1.4.1 (Tensor slice). Dato $T \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$ chiamiamo *slice* (o *fetta*) di T ogni matrice ottenuta da T fissando tutti gli indici tranne due. Le slices ottenute fissando gli ultimi $d-2$ indici sono dette *frontal slices*. Nel caso $d=3$ si identificano tutte le possibili slices:



Definizione 1.4.2 (Mode- k fiber). Dato un tensore $T \in \mathbb{R}^{n_1 \times \dots \times n_d}$ chiamiamo *fibers* i vettori ottenuti fissando tutti gli indici tranne uno. Quando l'indice libero di variare è il k -esimo lo chiamiamo *mode- k fiber*. Nel caso $d=3$ denotiamo le fibre come

Definizione fi-
bre

In molte situazioni risulta comodo disporre gli elementi di un tensore in una matrice. Uno dei modi per farlo è utilizzare le matricizzazioni, anche dette *unfoldings*.

Definizione 1.4.3 (Mode- k unfolding). Il *mode- k unfolding* di un tensore $T \in \mathbb{R}^{n_1 \times \dots \times n_d}$ è la matrice $\mathbf{T}_{(k)} \in \mathbb{R}^{n_k \times \prod_{j \neq k} n_j}$ le cui colonne sono mode- k fibers ordinate seguendo l'ordine naturale su $[n_1] \otimes \dots \otimes [n_{k-1}] \otimes [n_{k+1}] \otimes \dots \otimes [n_d]$. In particolare si ha

$$T(i, j) = T(i_1, \dots, i_{k-1}, i, i_{k+1}, \dots, i_d), \text{ con } j = \mathbb{L}(i_1, \dots, i_{k-1}, i_{k+1}, \dots, i_d).$$

L'operazione di unfolding permette anche di catturare il suo rango (matriciale) lungo ogni direzione:

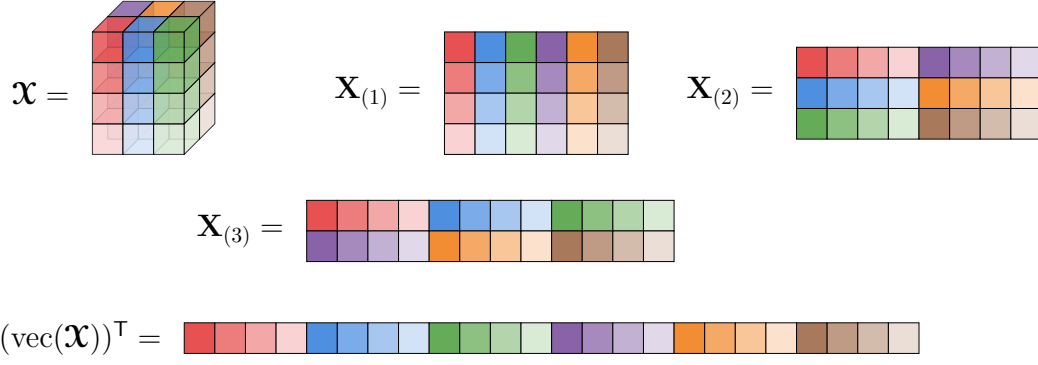


Figura 1.2: Schema delle matricizzazioni di un tensore.

Definizione 1.4.4 (Multirank). Il *multilinear rank* o *multirank* di un tensore $T \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$ è la d -upla $(r_1, r_2, \dots, r_d)$, dove $r_k = \text{rank } \mathbf{T}_{(k)}$, per ogni $k \in \{1, \dots, d\}$.

Dal punto di vista computazionale è utile introdurre la seguente operazione

Definizione 1.4.5 (Reshape). Definiamo l'operatore di *reshape* come l'operazione riarrangia il tensore senza alterarne le dimensioni e mantenendo l'ordine delle entrate. Formalmente, fissati $d, d' \in \mathbb{N}$ e un tensore $T \in \mathbb{R}^{n_1 \times \dots \times n_d}$, per ogni $m_1, \dots, m_{d'}$ tali che $\prod_{i=1}^{d'} m_i = \prod_{j=1}^d n_j$, il tensore $\mathbf{Y} = \text{RESHAPE}(T, m_1 \times \dots \times m_{d'}) \in \mathbb{R}^{m_1 \times \dots \times m_{d'}}$ è definito come

$$\mathbf{Y}(i_1, \dots, i_{d'}) = T(j_1, \dots, j_d),$$

per ogni $(i_1, \dots, i_{d'}), (j_1, \dots, j_d)$ tali che $\mathbb{L}(i_1, \dots, i_{d'}; m_1 \times \dots \times m_{d'}) = \mathbb{L}(j_1, \dots, j_d; n_1 \times \dots \times n_d)$.

Lemma 1.4.6 (Legame tra il prodotto esterno e il prodotto di Kroneker). *Dato un tensore $T = v^1 \circ \dots \circ v^d$, allora*

- (i) $\text{vec}(T) = v^d \otimes \dots \otimes v^1$
- (ii) $\mathbf{T}_{(k)} = v^k (v^d \otimes \dots \otimes v^{k-1} \otimes v^{k+1} \otimes \dots \otimes v^1)^\top$
- (iii) $\mathbf{T}_{(\mathcal{R} \times \mathcal{C})} = (v^{r_\delta} \otimes \dots \otimes v^{r_1})(v^{c_{d-\delta}} \otimes \dots \otimes v^{c_1})^\top$, dove $\mathcal{R} = \{r_1, \dots, r_\delta\}$ e $\mathcal{C} = \{c_1, \dots, c_{d-\delta}\}$.

In particolare, tutte le matricizzazioni di un prodotto esterno hanno rango 1.

Dimostrazione. La (i) segue applicando più volte l'Osservazione 1.2.5. Dimostriamo la (ii): □

refrasare un po-
co per renderla
più leggibile

In molte situazioni è utile moltiplicare un tensore con una matrice, ma purtroppo le dimensioni non combaciano. La seguente definizione risolve il problema facendo agire la matrice lungo il modo k -esimo di un tensore, e si chiama.

Definizione 1.4.7 (Prodotto nel modo k). Il *prodotto nel modo k* di un tensore $T \in \mathbb{R}^{n_1 \times \dots \times n_d}$ con una matrice $U \in \mathbb{R}^{r \times n_k}$ descrive l'azione di U sulla k -esima coordinata di T . Formalmente, è un tensore

$$\mathbf{Y} = T \times_k U \in \mathbb{R}^{n_1 \times \dots \times n_{k-1} \times r \times n_{k+1} \times \dots \times n_d} \text{ con } \mathbf{Y}_{(k)} = U \mathbf{T}_{(k)}.$$

In componenti, $\mathbf{Y}(i_1, \dots, i_{k-1}, \alpha, i_{k+1}, \dots, i_d) = \sum_{i_k=1}^d T(i_1, i_2, \dots, i_d) U_{\alpha, i_k}$.

Lemma 1.4.8. Siano $\mathbf{A} \in \mathbb{K}^{m \times n}$, $\mathbf{B} \in \mathbb{K}^{m \times n}$ ed $\mathbf{X} \in \mathbb{K}^{n \times m}$. Allora

$$\text{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes \mathbf{A}) \text{vec}(\mathbf{X})$$

Dimostrazione. Scrivendo $B = [b_1, \dots, b_n]$ ed $X = [x_1, \dots, x_n]$, la k -esima colonna di AXB è

$$(AXB)_{:,k} = AXb_k = A \sum_{i=1}^m x_i b_{i,k} = [b_{1,k}A, \dots, b_{n,k}A] \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} = ([b_{1,k}, \dots, b_{n,k}] \otimes A) \text{vec}(X).$$

Impilando insieme le colonne otteniamo che

$$\text{vec}(AXB) = \begin{bmatrix} (AXB)_{:,1} \\ (AXB)_{:,2} \\ \vdots \\ (AXB)_{:,n} \end{bmatrix} = \begin{bmatrix} b_1^\top \otimes A \\ b_2^\top \otimes A \\ \vdots \\ b_n^\top \otimes A \end{bmatrix} \text{vec}(X) = (\mathbf{B}^\top \otimes \mathbf{A}) \text{vec}(X).$$

□

Proposizione 1.4.9. Dato un tensore $T \in \mathbb{R}^{n_1 \times n_2 \times n_3}$ e delle matrici $U \in \mathbb{R}^{m_1 \times n_1}$, $V \in \mathbb{R}^{m_1 \times n_2}$ e $W \in \mathbb{R}^{m_3 \times n_3}$. Sono fatti equivalenti:

$$(i) \quad \mathbf{Y} = T \times_1 U \times_2 V \times_3 W \in \mathbb{R}^{m_1 \times m_2 \times m_3}.$$

$$(ii) \quad \text{vec}(\mathbf{Y}) = (W \otimes V \otimes U) \cdot \text{vec}(T).$$

$$(iii) \quad \mathbf{Y}_{(1)} = U \mathbf{T}_{(1)} (W \otimes V)^\top.$$

$$(iv) \quad \mathbf{Y}_{(2)} = V \mathbf{T}_{(2)} (W \otimes U)^\top.$$

$$(v) \mathbf{Y}_{(3)} = W \mathbf{T}_{(3)} (V \otimes U)^\top.$$

Dimostrazione. (1) $\iff$ (2) Mostriamo che $\text{vec}(T \times_1 U) = (I \otimes I \otimes U) \text{vec}(T)$.

(2) $\iff$ (4) Conversely, assume statement 2 holds. Thus...

□

Proposizione 1.4.10. Siano $\mathbf{A} \in \mathbb{R}^{m \times p}$ e $\mathbf{B} \in \mathbb{R}^{n \times q}$. Allora,

$$\mathbf{A} \otimes \mathbf{B} = \mathbf{P}(\mathbf{B} \otimes \mathbf{A})\mathbf{Q}^\top, .$$

dove $\mathbf{P}$ e $\mathbf{Q}$ sono matrici di permutazione (m, n) - e (p, q) - perfect shuffle, rispettivamente.

ADD

Dimostrazione.

□

Esempio polinomio semisep + altro es

Definizione 1.4.11 (Prodotto Khatri-Rao). Date due matrici $A \in \mathbb{R}^{m \times r}$ e $B \in \mathbb{R}^{n \times r}$, il loro *prodotto Khatri-Rao* è dato da

$$(1.3) \quad C = A \odot B = [a_1 \otimes b_1 \mid \dots \mid a_r \otimes b_r] \in \mathbb{R}^{(mn) \times r},$$

dove a_j e b_j indicano la j -esima colonna di A e B , rispettivamente, e la matrice C si lascia scrivere in componenti come $C_{k,l} = A_{i,l} B_{j,l}$, dove $\mathbb{L}^*(i, j) = \mathbb{L}(j, i) = k$.

Osservazione 1.4.12. Calcolare il prodotto di Khatri-Rao di due matrici $A \in \mathbb{R}^{m \times r}$ e $B \in \mathbb{R}^{n \times r}$ ha un costo in memoria di $O(mn \cdot r)$.

1.5 IL PROBLEMA DEL RANGO

non mi piace

In più dimensioni il problema di calcolare il rango, e una fattorizzazione CP di un tensore si complicano notevolmente rispetto al caso matriciale.

BORDER RANK

L'insieme dei tensori di un rango fissato r non è chiuso, ad esempio, un tensore di rango 3 può convergere ad uno di rango 2, come mostra l'Esempio 2.8.1. Questo limita l'utilità del rango dal punto di vista teorico, ma ha risvolti applicativi interessanti, che vedremo nella Sezione 2.8.

COSTO COMPUTAZIONALE

Calcolare il costo di una fattorizzazione CP usando un algoritmo esatto è NP-Hard [15]. Non solo; il problema di trovare un tensore approssimante di un certo rango è computazionalmente difficile. In altre parole, la miglior approssimazione di rango r è data da dei vettori $x_i \in \mathbb{R}^{n_1}, y_i \in \mathbb{R}^{n_2}, \dots, z_i \in \mathbb{R}^{n_d}$, per $i = 1, \dots, r$ che minimizzano

$$\|A - x_1 \otimes y_1 \otimes \dots \otimes z_1 - \dots - x_r \otimes y_r \otimes \dots \otimes z_r\|$$

oppure, in altre parole un tensore B dato da una fattorizzazione CP, che realizza

$$\min_{\text{rank}(B) \leq r} \|A - B\|.$$

Qui $\|\cdot\|$ denota una norma fissata su $\mathbb{R}^{n_1 \times \dots \times n_d}$. Quando $d = 2$, il problema è risolto per norme invarianti per trasformazioni unitarie su $\mathbb{R}^{m \times n}$ dal teorema di Eckart-Young-Mirsky [12]: se

$$A = U \Sigma V^* = \sum_{i=1}^{\text{rank}(A)} \sigma_i u_i \otimes v_i, \quad \sigma_i \geq \sigma_{i+1},$$

è la decomposizione ai valori singolari di A (anche detta SVD), allora la migliore approssimazione di rango r è data dai primi r termini della somma. Sfortunatamente, generalizzare questo risultato in più dimensioni è ritenuto molto difficile (o impossibile) [21].

RANGO TIPICO

Mentre le matrici random hanno rango massimo con probabilità uno, per un tensore di un formato fissato potrebbe esserci più di un rango con probabilità positiva. Questo fenomeno è noto come *rango tipico* e verrà discusso nella Sezione. why? ref

DIPENDENZA DAL CAMPO BASE

Nel caso matriciale ogni matrice $m \times n$ a entrate reali ha lo stesso rango se visto come elemento di $\mathbb{R}^{m \times n}$ o di $\mathbb{C}^{m \times n}$, per i tensori questo è falso. Consideriamo il seguente esempio:

Esempio 1.5.1 (De Silva and Lim [21]). Scriviamo $z_k = x_k + iy_k$ e $\bar{z}_k = x_k - iy_k$, allora

$$(1.4) \quad \begin{aligned} x_1 \circ x_2 \circ x_3 + x_1 \circ y_2 \circ y_3 - y_1 \circ x_2 \circ y_3 + y_1 \circ y_2 \circ x_3 \\ = \frac{1}{2}(\bar{z}_1 \circ z_2 \circ \bar{z}_3 + z_1 \circ \bar{z}_2 \circ z_3) \end{aligned}$$

espandi conti e aggiungere considerazioni su svd da articolo

1.6 IL TEOREMA DEL RANGO

Data l'infattibilità del problema del rango, possiamo dare una stima del rango con la seguente proposizione, dalla quale intuimmo che non sarà semplice trovare stime del rango di un tensore—queste spesso coinvolgono tecniche algebrico-combinatorie avanzate. Vedremo alcuni esempi di queste stime nella Sezione 2.5.

Proposizione 1.6.1 (Teorema del rango). *Sia $T \in \mathbb{R}^{n_1 \times \dots \times n_d}$ un tensore che ammette fattorizzazione $T = \llbracket U_1, \dots, U_d \rrbracket$, e sia $r_j = \mathbf{R}(X_{(j)})$ il rango dell'unfolding $X_{(j)}$, con $j = 1, \dots, d$. Allora vale che*

$$(1.5) \quad \max_{j=1, \dots, d} (r_j) \leq \mathbf{R}(T) \leq \min_{j=1, \dots, d} \left(\prod_{i \neq j} r_i \right)$$

Introduciamo un lemma di supporto:

Lemma 1.6.2. *Data una fattorizzazione $T = \llbracket U_1, \dots, U_d \rrbracket \in \mathbb{R}^{n_1 \times \dots \times n_d}$, essa ha come unfoldings*

$$\mathbf{T}_{(j)} = U_j(U_d \odot U_{d-1} \cdots \odot U_{j+1} \odot U_{j-1} \odot \cdots \odot U_1)^\top,$$

per ogni $j = 1, \dots, d$.

Dimostrazione. Scrivendo le matricizzazioni di $T = \sum_{h=1}^r u_h^1 \odot \dots \odot u_h^d$, dove i vettori $u_1^j, \dots, u_r^j$ rappresentano le colonne della matrice U_j , troviamo che

$$\begin{aligned} \mathbf{T}_{(j)} &= \sum_{h=1}^r u_h^j \left(u_h^d \otimes \dots \otimes u_h^{j+1} \otimes u_h^{j-1} \otimes \dots \otimes u_h^1 \right)^\top \\ &= U_j \left[u_1^d \otimes \dots \otimes u_1^1 \mid \dots \mid u_r^d \otimes \dots \otimes u_r^1 \right]^\top \\ &= U_j(U_d \odot \dots \odot U_{j+1} \odot U_{j-1} \odot \dots \odot U_1)^\top \end{aligned}$$

primo passaggio non chiaro

□

Dimostrazione del teorema del rango. La minorazione segue direttamente dalla scrittura delle ?? Mostriamo la maggiorazione, assumendo senza perdita di generalità che $\mathbf{R}(T) \leq \prod_{i=2}^d r_i$. Consideriamo la vettorizzazione

$$(1.6) \quad \text{vec}(T) = \sum_{k=1}^r u_k^d \otimes \cdots \otimes u_k^2 \otimes u_k^1.$$

Allora l'insieme

$$\{u_k^d \otimes \cdots \otimes u_k^2\}$$

è formato da vettori linearmente indipendenti, per ogni $k = 1, \dots, r$. Se così non fosse, esisterebbe un indice k_0 , che possiamo supporre $k_0 = 1$ a meno di riordinamento degli indici k , tale per cui

$$(1.7) \quad u_1^d \otimes \cdots \otimes u_1^2 = \sum_{h=2}^r c_h (u_h^d \otimes \cdots \otimes u_h^2).$$

che sostituita nella Equazione 1.6 porta alla seguente decomposizione:

$$\begin{aligned} \text{vec}(T) &= \sum_{k=2}^r u_k^d \otimes \cdots \otimes u_k^2 \otimes u_k^1 + \left(\sum_{k=2}^r c_k (u_k^d \otimes \cdots \otimes u_k^2) \right) \otimes u_1^1, \\ &= \sum_{k=2}^r (u_k^d \otimes \cdots \otimes u_k^2) \otimes (u_k^1 + c_k u_1^1) \end{aligned}$$

che ha rango $r - 1$, contraddicendo la minimalità di r . Quindi per il lemma (! missing) avremmo che

$$\mathbf{T}_{(1)} = U_1(U_d \odot \cdots \odot U_2)^\top,$$

dove $\mathbf{R}(U_d \odot \cdots \odot U_2) = r$. Siccome $U_d \odot \cdots \odot U_r$ è sottomatrice di $U_d \otimes \cdots \otimes U_2$, abbiamo dal lemma sul rango del krp (missing) che

$$r \leq \mathbf{R}(U_d \otimes \cdots \otimes U_2) = r_2 \cdots r_d.$$

Ripetendo l'argomento per gli altri unfoldings si ha la tesi. □

Ci sono molte domande aperte riguardo al rango, ad esempio non si ha una risposta generale (a differenza del caso matriciale) a domande del tipo:

1. Qual'è il massimo rango possibile per un tensore di dimensione $n_1 \times \cdots \times n_d$?

1 La fattorizzazione CP

2. Quali sono i ranghi *tipici*, ovvero quelli ottenuti con probabilità non nulla quando si genera un tensore di entrate casuali?

1.7 RANGO TIPICO

Riflettendo sulla Proposizione 1.6.1 possiamo immaginare che stimare il rango di un tensore sia un problema complesso.

Explain better

1.8 OPERAZIONI NEL FORMATO CP

Se serve, versione per unfoldings generalizzati

Lemma 1.8.1. Dato un tensore $T = \llbracket U_1, \dots, U_d \rrbracket$, vale $\text{vec}(T) = (U_d \odot U_{d-1} \odot \dots \odot U_1) \mathbf{1}_r$, dove $\mathbf{1}_r = [1, \dots, 1]^\top$.

Dimostrazione. Segue dall'identità

$$\begin{aligned} \text{vec}(T) &= \sum_{h=1}^r u_h^d \otimes \dots \otimes u_h^1 = \left[u_1^d \otimes \dots \otimes u_1^1 \mid \dots \mid u_r^d \otimes \dots \otimes u_r^1 \right]^\top \mathbf{1}_r \\ &= (U_d \odot U_{d-1} \odot \dots \odot U_1) \mathbf{1}_r. \end{aligned}$$

□

1.9 CALCOLO DELLA FATTORIZZAZIONE CP

Trattiamo il problema di trovare la migliore approssimazione con rango r di un dato tensore. Studiamo il caso $d = 3$. Dato $T \in \mathbb{R}^{m \times n \times p}$ cerchiamo una fattorizzazione $\llbracket A, B, C \rrbracket$ con $A \in \mathbb{R}^{m \times r}$, $B \in \mathbb{R}^{n \times r}$ e $C \in \mathbb{R}^{p \times r}$ cercando di risolvere il problema ai minimi quadrati

$$(1.8) \quad \min_{A, B, C} \|T - \llbracket A, B, C \rrbracket\|^2.$$

Tale problema può essere mal posto per quanto detto sul border rank. Inoltre si tratta di un problema non convesso (esistono molti minimi locali) e dato che ogni tensore ha rappresentazioni CP multiple.

why?

ALTERNATING LEAST-SQUARES

Considero

$$\|T - \llbracket A, B, C \rrbracket\|^2 = \|\mathbf{T}_{(1)} - A(C \odot B)^\top\|_F^2 = \|(C \odot B)A^\top - \mathbf{T}_{(1)}^\top\|_F^2,$$

ovvero un problema lineare ai minimi quadrati dove l'incognita è una matrice A . La soluzione può essere espressa in termini delle equazioni normali

why?

$$(C \odot B)^\top (C \odot B) A^\top = (C \odot B)^\top \mathbf{T}_{(1)}^\top,$$

se e solo se, usando il lemma

lemma KRP
HADamard

$$(C^\top C * B^\top B) A^\top = (C \odot B)^\top \mathbf{T}_{(1)}^\top.$$

La matrice $Z = \mathbb{R}^{r \times r}$ è semidefinita positiva, e quando è positiva definita (accade se $C \odot B$ è full-rank possiamo risolvere con la fattorizzazione di Cholesky il sistema)

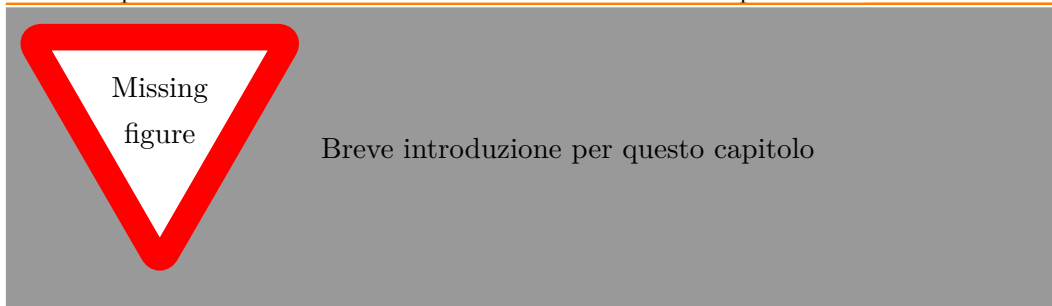
$$(1.9) \quad AZ = \mathbf{T}_{(1)}(C \odot B) = M$$

vEdi fogli per
calcolo costo

2

APPLICAZIONI DELLA CP ALLA COMPLESSITÀ ARITMETICA

Conoscere il rango CP di tensori che rappresentano applicazioni multilineari può rivelare informazioni sulla complessità asintotica e fornire algoritmi con complessità ottimale per la loro valutazione. Proviamo a formulare il problema.



sistemare

Controlla che non ci sia ambiguità nel riferirti a mappe/forme bilineari: $\psi : U \times V \rightarrow W$ è una forma bilineare se $W = \mathbb{K}$, altrimenti chiamala mappa bilineare

rinomina le forme bilineari con φ invece che f

Fix conflicting notation!

2.1 MAPPE MULTILINEARI E MOLTIPLICAZIONE TRA MATRICI

Siano M, N, P spazi vettoriali di dimensione finita, di dimensione m, n, p rispettivamente. Una forma bilineare è una mappa $f : M \times N \rightarrow P$ lineare in ogni componente, in altre parole tale che $f(m, \cdot)$ e $f(\cdot, n)$ sono lineari per ogni $m \in M$ e $n \in N$. In maniera analoga si definiscono le mappe trilineari $f : M_1 \times M_2 \times M_3 \rightarrow P$.

Definizione 2.1.1 (Moltiplicazione matriciale). Fissati degli interi positivi m, n, p , la moltiplicazione tra due matrici $m \times n$ e $n \times p$ è una mappa bilineare

$$\langle m, n, p \rangle : \mathbb{C}^{m \times n} \times \mathbb{C}^{n \times p} \rightarrow \mathbb{C}^{m \times p}$$

$$(A, B) \mapsto C = AB.$$

Spesso ci riferiremo a $\langle m, n, p \rangle$ chiamandolo "algoritmo", ad esempio nel contesto degli algoritmi di moltiplicazione veloce.

Osservazione 2.1.2. La bilinearità della mappa sopra segue immediatamente dalla definizione

$$(2.1) \quad (AB)_{ij} = \sum_{k=1}^N a_{ik}b_{kj},$$

dove $i = 1, \dots, M$ e $j = 1, \dots, P$.

Osservazione 2.1.3. Possiamo vedere la moltiplicazione tra matrici anche come una mappa trilineare

$$\mathbb{C}^{M \times N} \times \mathbb{C}^{N \times P} \times \mathbb{C}^{M \times P} \rightarrow \mathbb{C}$$

$$(A, B, C) \mapsto \text{trace}(ABC)$$

giustifica se
necessario

se riesci muovi
al capitolo 1

RAPPRESENTAZIONE IN COORDINATE DI UNA FORMA BILINEARE

Sfruttando l'isomorfismo con lo spazio vettoriale standard, possiamo rappresentare una forma bilineare $f: X \times Y \rightarrow Z$ come un tensore nel seguente modo: fissato $k \in \{1, \dots, p\}$ e considerata $f_k: X \times Y \rightarrow \mathbb{R}$, esiste una matrice $M \in \mathbb{R}^{m \times n}$ tale che $f_k(x, y) = x^\top M y$ per ogni $(x, y) \in \mathbb{R}^m \times \mathbb{R}^n$. Scrivendo ogni componente di

$$f = \begin{bmatrix} f_1 \\ \vdots \\ f_p \end{bmatrix} \text{ in questa forma, otteniamo un tensore } T \in \mathbb{R}^{m \times n \times p} \text{ tale che le sue slice}$$

frontali rappresentino ognuna delle p forme bilineari f_k , ossia che

$$(2.2) \quad f_k(x, y) = x^\top T_{:, :, k} y = \sum_{i=1}^m \sum_{j=1}^n T_{i,j,k} x_i y_j, \text{ per ogni } k = 1, \dots, p.$$

In forma tensoriale,

$$(2.3) \quad f(x, y) = \begin{bmatrix} f_1(x, y) \\ \vdots \\ f_p(x, y) \end{bmatrix} = \begin{bmatrix} x^\top T_{:, :, 1} y \\ \vdots \\ x^\top T_{:, :, p} y \end{bmatrix} = T \times_1 x^\top \times_2 y^\top.$$

Osservazione 2.1.4. Questa procedura si estende (passando alle coordinate) anche al caso in cui le entrate di x e y siano oggetti algebrici più complessi di scalari, ad esempio vettori o matrici.

2.2 L'ESPONENTE DELLA MOLTIPLICAZIONE TRA MATRICI

In questo capitolo useremo il seguente modello computazionale per il calcolo della complessità, per calcolare costi computazionali anche su campi infiniti, e semplificare il calcolo dei costi eliminando branching (e.g. `if-then-else` o `while`).

Definizione 2.2.1 (Straight line program). Uno *straight line program* (o *circuito algebrico*) Γ su $\mathbb{K}$ è una sequenza ordinata di istruzioni della forma

$$(\mathbf{op}; i, j) \text{ oppure } (\lambda; i)$$

dove $\mathbf{op} \in \{+, -, *, /\}$ è una delle quattro operazioni aritmetiche fondamentali (binarie) e $\lambda \in \mathbb{K}$. Il primo tipo di istruzioni descrive l'esecuzione dell'operazione aritmetica $\mathbf{op}$ fra gli i -esimi e j -esimi risultati intermedi, mentre il secondo tipo descrive la moltiplicazione dell' i -esimo risultato intermedio con lo scalare λ .

Definizione 2.2.2 (Funzione costo). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali di dimensione finita. Definiamo la *funzione costo* associata a uno straight line program Γ come

$$C_\Gamma(\varphi) := \text{numero di operazioni } \{*, /\} \text{ per calcolare } \varphi \text{ sul circuito } \Gamma, \text{ e}$$

$$C_\Gamma^{\text{tot}}(\varphi) := \text{numero di operazioni } \{+, -, *, /\} \text{ per calcolare } \varphi \text{ sul circuito } \Gamma.$$

Definizione 2.2.3 (Funzione di complessità aritmetica). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali di dimensione finita. Definiamo

$$L^{\text{tot}}(\varphi) := \inf_{\Gamma \text{ straight line program}} \{C_\Gamma^{\text{tot}}(\varphi)\} \quad \text{la complessità totale di } \varphi, \text{ e}$$

$$L(\varphi) := \inf_{\Gamma \text{ straight line program}} \{C_\Gamma(\varphi)\} \quad \text{la complessità moltiplicativa di } \varphi.$$

Osservazione 2.2.4. In altre parole, $L(\varphi)$ è il minimo numero di moltiplicazioni non scalari sufficienti a calcolare φ , anche dette *active multiplication*. In maniera analoga, L^{tot} è il minimo numero di moltiplicazioni non scalari e di addizioni sufficienti a calcolare φ .

Consideriamo la forma bilineare $\varphi = \langle n, n, n \rangle$ data dall'Equazione 2.1. Fissato un campo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, e delle matrici $A, B \in \mathbb{R}^{n \times n}$, denotiamo con

$$(2.4) \quad M_{\mathbb{K}}(n) := L^{\text{tot}}(\langle n, n, n \rangle) = L^{\text{tot}}\left(\left\{\sum_{k=1}^n a_{ik}b_{kj} \mid i, j \in \{1, \dots, n\}\right\}\right)$$

specifichiamo a che spazio appartengono i risultati intermedi?

il numero totale di moltiplicazioni aritmetiche sufficienti a calcolare la moltiplicazione di due matrici $n \times n$. Determinare una formula per $M_{\mathbb{K}}(n)$ è lontano dagli strumenti attualmente disponibili. Invece, il problema di approssimare la crescita asintotica della funzione $n \mapsto M_{\mathbb{K}}(n)$ è fattibile; definiamo

Definizione 2.2.5. L'esponente $\omega_{\mathbb{K}}$ di una moltiplicazione matriciale è il numero

$$\omega_{\mathbb{K}} := \inf_n \{ \tau \in \mathbb{R} \mid \text{la moltiplicazione tra due matrici in } \mathbb{K}^{n \times n} \text{ ha costo } o(n^\tau) \text{ operazioni aritmetiche} \}.$$

In altre parole, $\omega_{\mathbb{K}} = \inf \{ \tau \in \mathbb{R} \mid M_{\mathbb{K}}(n) = o(n^\tau) \}$.

Osservazione 2.2.6. Dalla notazione $M_{\mathbb{K}}(h)$ e $\omega_{\mathbb{K}}$ è dovuta alla dipendenza dal campo $\mathbb{K}$. Dato che tratteremo solo $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, spesso scriveremo ω per $\omega_{\mathbb{K}}$.

Teorema 2.2.7 (Schönhage [20]). *L'esponente della moltiplicazione tra matrici è invariante rispetto alle estensioni scalari: se $\mathbb{K} \subset \mathbb{L}$ è un'estensione di campo, allora*

$$\omega_{\mathbb{K}} = \omega_{\mathbb{L}}$$

Osserviamo che $M_{\mathbb{K}}(n) \leq n^2(2n-1)$. Siccome il costo computazionale dev'essere almeno pari al numero di entrate della matrice, vale $M_{\mathbb{K}}(n) \geq n^2$, da cui $\omega \in [2, 3]$. Trovare ω è un problema centrale nella teoria della complessità, e si congettura che $\omega = 2$, cioè che il costo della moltiplicazioni possa essere asintoticamente pari a quello delle moltiplicazioni. L'algoritmo naive ha $\omega = 3$, mentre il primo miglioramento, dovuto all'algoritmo di Strassen [25] mostra che $\omega \leq \log_2(7) < 2.81$. Il record attuale è $\omega \leq 2.371177$ [2]. Dopo Strassen, nel 1978 i miglioramenti significativi sono stati $\omega \leq 2.7962$ ad opera di [19], poi $\omega \leq 2.7799$ [4] nel 1979, poi $\omega \leq 2.55$ [20], poi $\omega \leq 2.48$ nel 1987 [24], e poi $\omega \leq 2.38$ [8] nel 1989, che potrebbe aver fatto pensare che nel 1990 si sarebbe trovato il bound. Dopo tale miglioramento, l'approssimazione è scesa molto lentamente, fino a scendere sotto $\omega < 2.372$ solo recentemente [1, 14, 23, 26]. Tuttavia, l'avvento dell'Intelligenza Artificiale sta dando un nuovo impulso a questa ricerca, scoprendo nuovi algoritmi esatti per matrici di piccole dimensioni (si veda ad esempio AlphaTensor [13]); di recente (agosto 2026) l'utilizzo di strumenti di IA come AlphaEvolve ha stabilito il nuovo record di $\omega \leq 2.371177$ [2].

Osservazione 2.2.8. Nella maggior parte delle applicazioni, l'algoritmo di Strassen è quello ottimale, si veda la discussione in [10]. Di recente, il problema di stimare ω e trovare il rango di tensori è diventato un problema d'interesse della branca della teoria della complessità algebrica e della geometria algebrica. Si veda ad esempio [18].

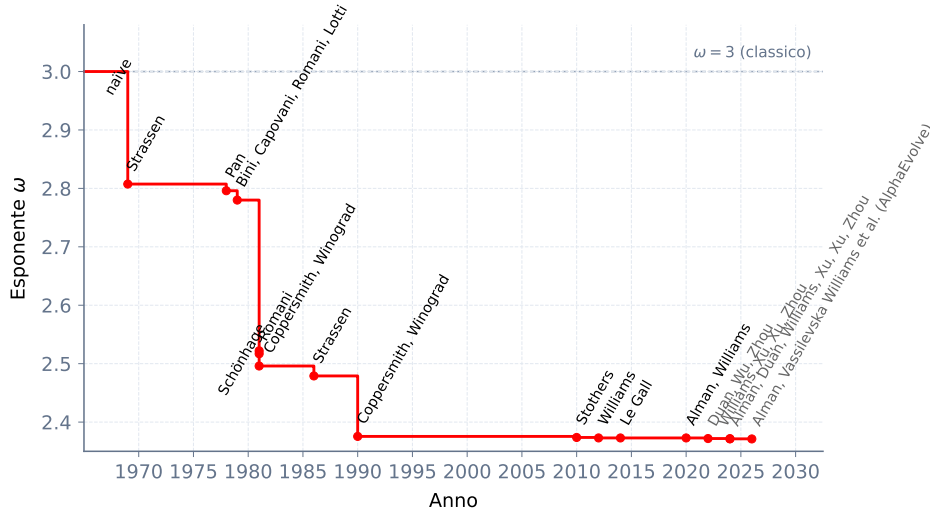


Figura 2.1: Miglioramenti del limite superiore dell'esponente della moltiplicazione matriciale ω dal 1969 ad oggi. La linea tratteggiata rappresenta il limite superiore $\omega \leq 3$.

2.3 DECOMPOSIZIONE LOW-RANK DI FORME BILINEARI

Nel calcolo della complessità, le decomposizioni low-rank possono ridurre notevolmente le operazioni aritmetiche sulle matrici. Se ad esempio $M \in \mathbb{R}^{m \times n}$ è una matrice di rango uno, possiamo scrivere $M = u \circ v = uv^\top$ per certi $u \in \mathbb{R}^m$ e $v \in \mathbb{R}^n$; valutare $Mx = (uv^\top)x = u(v^\top x)$ ha costo $O(n)$, contro il costo $O(mn)$ del prodotto classico matrice vettore. In generale, se M ha rango R , possiamo scriverla come somma di R matrici di rango uno ed ottenere un costo $O(Rn)$ per Mx . È possibile riadattare quest'idea per forme bilineari, come vedremo ora.

Calcolare la valutazione di una forma bilineare f come nell'Equazione 2.2 ha un costo $O(mnp)$, data dal numero di moltiplicazioni necessarie fra gli elementi di x e y . Queste sono dette *active multiplications* e corrispondono al numero di elementi non nulli di T . Ci aspettiamo che la complessità di valutare $f(x, y)$ sia correlata al costo delle active multiplications. Cerchiamo di minimizzarne il numero cercando fattorizzazioni CP di T .

Definizione 2.3.1 (Bilinear algorithm). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare tra $\mathbb{K}$ -spazi vettoriali. Per ogni $i \in \{1, \dots, r\}$, siano $f_i \in U^*$, $g_i \in V^*$, $w_i \in W$ tali che

$$\varphi(u, v) = \sum_{i=1}^r f_i(u)g_i(v)w_i,$$

Spostare davanti alla CP per alleggerire (già scritta)

aggiungere thm?

per ogni $u \in U, v \in V$. Allora la r -upla $(f_1, g_1, w_1; \dots; f_r, g_r, w_r)$ viene detta *bilinear computation (algorithm)* di lunghezza r per φ , e la lunghezza di più breve algoritmo di calcolo per φ è chiamata *bilinear complexity* o rango di φ , che indichiamo ancora con $\mathbf{R}(\varphi)$.

Proposizione 2.3.2 (Strassen). *Siano U, V e W dei $\mathbb{K}$ -spazi vettoriali finiti, e $\varphi : U \times V \rightarrow W$ una mappa bilineare. Allora*

$$L(\varphi) = \text{Lunghezza della più breve bilinear computation per } \varphi.$$

Osservazione 2.3.3. Da questa proposizione segue direttamente che

$$(2.5) \quad L(\varphi) \leq \mathbf{R}(\varphi) \leq 2L(\varphi).$$

La prossima definizione assicura che il rango di una forma bilineare è dato dal rango CP.

Proposizione 2.3.4. *Fissata una forma bilineare φ , e detto T il tensore che la rappresenta. Allora, dalla fattorizzazione CP di un tensore T si ottiene una bilinear computation, e in particolare $\mathbf{R}(\varphi) = \mathbf{R}(T)$.*

Dimostrazione. Data una decomposizione $T = \llbracket U, V, W \rrbracket$ di rango r , dalla Definizione 1.3.1 abbiamo che $T = \sum_{l=1}^r u_l \circ v_l \circ w_l$, o in componenti, $T_{i,j,k} = \sum_{l=1}^r u_{il} v_{jl} w_{kl}$. Inserendo questa relazione nell'Equazione 2.2 troviamo,

$$(2.6) \quad \begin{aligned} \varphi_k(x, y) &= \left(\left(\sum_{l=1}^r u_l \circ v_l \circ w_l \right) \times_1 x^\top \times_2 y^\top \right)_k = \left(\sum_{l=1}^r (u_l \circ v_l \circ w_l) \times_1 x^\top \times_2 y^\top \right)_k \\ &= \sum_{i=1}^m \sum_{j=1}^n \sum_{l=1}^r u_{il} v_{jl} w_{kl} x_i y_j = \sum_{l=1}^r \left(\sum_{i=1}^m u_{il} x_i \right) \cdot \left(\sum_{j=1}^n v_{jl} y_j \right) w_{kl} \\ &= \sum_{l=1}^r (u_l^\top x) \cdot (v_l^\top y) w_{kl}, \end{aligned}$$

per ogni $k = 1, \dots, p$. Mettendo insieme le Equazioni 2.3 e 2.6 troviamo

$$(2.7) \quad \varphi(x, y) = \sum_{l=1}^r (u_l^\top x) \cdot (v_l^\top y) w_l.$$

Considerando i funzionali $f_l(x) := u_l^\top x$ e $g_l(y) := v_l^\top y$ indotti dal prodotto scalare standard si ha la tesi.

□

Quindi valutare φ richiede esattamente $r = \mathbf{R}(T)$ active multiplications (e $O(r)$ addizioni), che sopra evidenziamo con $(\cdot)$. In particolare, vale il seguente risultato algebrico:

fix ora che è
cambiato sopra

Proposizione 2.3.5 (Si veda [7], Cap. 14). *Siano U, V e W dei $\mathbb{K}$ -spazi vettoriali. Allora esiste un unico isomorfismo $U^* \otimes V^* \otimes W \rightarrow \{\varphi : U \times V \rightarrow W, \text{ bilineare}\}$ che manda $f \otimes g \otimes w$ nella mappa bilineare $(u, v) \mapsto f(u)g(v)w$.*

Definizione 2.3.6 (Tensore strutturale). L'unico tensore $T \in U^* \otimes V^* \otimes W$ associato alla mappa bilineare $\varphi : U \times V \rightarrow W$ è detto *tensore strutturale* di φ .

2.4 RAPPRESENTAZIONE DEL PRODOTTO TRA MATRICI

Consideriamo due matrici $A \in \mathbb{R}^{M \times N}$ e $B \in \mathbb{R}^{N \times P}$ e il loro prodotto $C \in \mathbb{R}^{M \times P}$. Ponendo $x = \text{vec}(A)$ e $y = \text{vec}(B)$, l'Equazione 2.3 diventa

$$(2.8) \quad \text{vec}(C^\top) = \varphi(x, y) = T \times_1 \text{vec}(A)^\top \times_2 \text{vec}(B)^\top,$$

dove abbiamo fissato l'ordinamento naturale sulle entrate di $\text{vec}(A)$ e $\text{vec}(B)$, e quello inverso su $\text{vec}(C^\top)$. Da ciò troviamo un tensore $T \in \mathbb{R}^{MN \times NP \times MP}$ che rappresenta la moltiplicazione tra $\langle M, N, P \rangle$. Data una decomposizione CP di rango r del tensore $T = \llbracket U, V, W \rrbracket$, grazie all'Equazione 2.7, l'espressione sopra si traduce in

$$(2.9) \quad \text{vec}(C^\top) = \sum_{l=1}^r (u_l^\top \text{vec}(A)) \cdot (v_l^\top \text{vec}(B)) w_l = \sum_{l=1}^r (s_l \cdot t_l) w_l = \sum_{l=1}^r m_l w_l,$$

Dove abbiamo posto $s_l := s_l(A) := u_l^\top \text{vec}(A)$, $t_l := t_l(B) := v_l^\top \text{vec}(B)$ e $m_l := s_l t_l$. Nella pratica, nei passaggi successivi dell'algoritmo calcoliamo in maniera ricorsiva il prodotto tra s_l e t_l . Questa formula definisce un algoritmo di moltiplicazione veloce $\langle M, N, P \rangle$.

2.5 ALCUNE STIME SULL'ESPONENTE

Dal ragionamento sopra, $\mathbf{R}(\langle n, n, n \rangle)$ è dato dal numero di active multiplication nella molitiplicazione di due matrici, e quindi ne misura la complessità algoritmica. Possiamo quindi riscrivere ω come

$$(2.10) \quad \omega = \lim_{n \rightarrow +\infty} \log_n(\mathbf{R}(\langle n, n, n \rangle)).$$

FIX errore nella formula

Ad esempio, l'algoritmo di Strassen [25] mostra che $\mathbf{R}(\langle 2, 2, 2 \rangle) \leq 7$, e come dimostrato da Winograd [27], $\mathbf{R}(\langle 2, 2, 2 \rangle) = 7$, ovvero non è possibile moltiplicare due matrici 2×2 con meno di sette moltiplicazioni. Nei casi diversi da $\langle 2, 2, 2 \rangle$ il problema si complica, e attualmente abbiamo alcune stime più lasche: già per matrici 3×3 , sappiamo solo che $19 \leq \mathbf{R}(\langle 3, 3, 3 \rangle) \leq 23$ [6, 16], mentre la migliore stima astintotica dal basso per il caso di matrici $n \times n$ è $\frac{5}{2}n^2 - 3n \leq \mathbf{R}(\langle n, n, n \rangle)$ [5]. Si veda il capitolo 11 di [18] per ulteriori dettagli e stime.

TRATTARE IL CASO GENERALE

Date due matrici $n^i \times n^i$, possiamo vedere la loro moltiplicazione come la moltiplicazione di matrici $n \times n$ sull'algebra delle matrici $n^{i-1} \times n^{i-1}$, suddividendole in blocchi. Questo dice anche che, se $\mathbf{R}(\langle n, n, n \rangle) \leq r$, allora $\mathbf{R}(\langle n^i, n^i, n^i \rangle) \leq r^i$; infatti, combinando le prossime Proposizioni 2.5.3 e 2.5.4, e usando che il rango è invariante per isomorfismo, troviamo

$$\mathbf{R}(\langle n^i, n^i, n^i \rangle) = \mathbf{R}(\langle n, n, n \rangle^{\otimes i}) \leq \mathbf{R}(\langle n, n, n \rangle)^i \leq r^i.$$

Quindi, possiamo applicare ricorsivamente l'algoritmo di moltiplicazione veloce, che è disegnato per matrici $n \times n$, per ottenere la moltiplicazione di matrici $n^i \times n^i$ impiegando solo r^i moltiplicazioni.

Se invece $n > n_0$ sono due interi positivi, dato un algoritmo con un caso base fissato $\langle n_0, n_0, n_0 \rangle$, è possibile ottenerne uno per $\langle n, n, n \rangle$, con $n > n_0$, allargando la

matrice aggiungendo zeri fino alla dimensione $n_0^{\lceil \log_{n_0} n \rceil}$. La complessità dipenderà comunque da n_0 e da r , in quanto

$$\begin{aligned} \mathbf{R}(\langle n, n, n \rangle) &\leq \mathbf{R}(\langle n_0^{\lceil \log_{n_0} n \rceil}, n_0^{\lceil \log_{n_0} n \rceil}, n_0^{\lceil \log_{n_0} n \rceil} \rangle) && \text{dalla 2.5.2,} \\ &\leq \mathbf{R}(\langle n_0, n_0, n_0 \rangle)^{\lceil \log_{n_0} n \rceil} && \text{dalle 2.5.3 e 2.5.4,} \\ &\leq r^{\lceil \log_{n_0} n \rceil} && \text{per ipotesi } \mathbf{R}(\langle n, n, n \rangle) \leq r, \\ &\leq r \cdot n^{\log_{n_0} r} && \text{usando che } \lceil x \rceil \leq x + 1. \end{aligned}$$

Riassumendo, $\mathbf{R}(\langle n, n, n \rangle) \leq r \cdot n^{\log_{n_0} r}$. Dunque ignorando la costante e dall'Equazione 2.10, $\omega \leq \log_{n_0} r$. Ad esempio, per l'algoritmo di Strassen poniamo $r = 7, n_0 = 2$, da cui $\omega \leq \log_2 7 < 2.81$. Abbiamo dimostrato la seguente:

Proposizione 2.5.1. *Se $\mathbf{R}(\langle n, n, n \rangle) \leq r$ per degli interi positivi n, r , allora $n^\omega \leq r$.*

Nei passaggi sopra abbiamo anche osservato che se $n \leq n'$ e dato un algoritmo per n' , possiamo ottenerne uno per n allargando la matrice. Possiamo racchiudere questa proprietà nel seguente risultato:

Proposizione 2.5.2 (Monotonia del rango di un algoritmo). *Il rango di un algoritmo è monotono crescente rispetto alla dimensione, ovvero dati due interi positivi $n \leq n'$, allora $\mathbf{R}(\langle n, n, n \rangle) \leq \mathbf{R}(\langle n', n', n' \rangle)$.*

Riportiamo le proposizioni utilizzate per dimostrare la proposizione sopra, dimostrate in [7], Cap. 14,15.

Proposizione 2.5.3. *Siano e, e', h, h', l, l' degli interi positivi. Allora abbiamo*

$$\langle e, h, l \rangle \otimes \langle e', h', l' \rangle \simeq \langle ee', hh', ll' \rangle,$$

cioè le due forme bilineari sono isomorfe.

Proposizione 2.5.4. *Siano φ_1, φ_2 due mappe bilineari. Allora $\mathbf{R}(\varphi_1 \otimes \varphi_2) \leq \mathbf{R}(\varphi_1)\mathbf{R}(\varphi_2)$.*

2.6 IL RANGO COME MISURA DELLA COMPLESSITÀ

La prossima proposizione mostra che il rango di un tensore è una misura ottimale della complessità. Inoltre risponde all'esistenza del problema inverso: dato un intero

positivo r , è possibile trovare un algoritmo con r moltiplicazioni? La risposta precisa è data dal seguente risultato.

Teorema 2.6.1 (Strassen [25], vedi anche [7] §15.1).

$$\omega = \inf\{\tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Ovvero, è possibile moltiplicare due matrici $n \times n$ usando $O(n^{\omega+\varepsilon})$ operazioni aritmetiche se e solo se il rango tensoriale di $\langle n, n, n \rangle$ è $O(n^{\omega+\varepsilon})$.

Introduciamo alcuni strumenti utili a dimostrare questo risultato.

Definizione 2.6.2 (Mappa bilineare concise). Sia $\varphi : U \times V \rightarrow W$ una mappa bilineare. Diremo che φ è *concise* se valgono tutte e tre le condizioni sotto:

1. $\text{Ker}_L(\varphi) = \{u \in U \mid \varphi(u, \cdot) = 0\} = \{0\}$,
2. $\text{Ker}_R(\varphi) = \{v \in V \mid \varphi(\cdot, v) = 0\} = \{0\}$,
3. $\text{Im}(\varphi) = \{\varphi(u, v) \mid u \in U, v \in V\} = W$.

change to roman and to i -concise per matchare quella dei tensori

Osservazione 2.6.3.

Esempio 2.6.4. La mappa bilineare $\langle n, n, n \rangle$ è concise.

Lemma 2.6.5. Sia $x = (x_0, x_1, x_2, \dots)$ una successione di numeri reali che soddisfa la ricorsione $x_s = ax_{s-1} + cb^s$, per certi numeri reali a, b, c . Allora per ogni $s \geq 0$, vale

$$x_s = \begin{cases} a^s x_0 + (a^s - b^s) \frac{bc}{a-b} & \text{se } a \neq b, \\ (cs + x_0)a^s & \text{se } a = b. \end{cases}$$

Dimostrazione. Mostriamo la formula per induzione su s . Per $s = 0$ la formula è verificata.

$$\begin{aligned} x_1 &= ax_0 + bc, \\ x_2 &= a(ax_0 + bc) + b^2c \\ &= a^2x_0 + (a+b)bc \\ &= \begin{cases} a^2x_0 + (a^2 - b^2) \frac{bc}{a-b} & \text{se } a \neq b, \\ (2c + x_0)a^2 & \text{se } a = b. \end{cases} \end{aligned}$$

Assumiamo la proprietà vera per $n-1$ e mostriamola per n .

osservare che combacia con la def data per i tensori

Se $a \neq b$,

$$\begin{aligned} x_s &= a \left(a^{s-1} x_0 + (a^{s-1} - b^{s-1}) \frac{bc}{a-b} \right) + b^s c \\ &= a^s x_0 + ((a^{s-1} - b^{s-1})a + b^{s-1}(a-b)) \frac{bc}{a-b} \\ &= a^s x_0 + (a^s - b^s) \frac{bc}{a-b} \end{aligned}$$

Similmente, se $a = b$

$$\begin{aligned} x_s &= (c(s-1) + x_0 + c)a^s \\ &= (cs + x_0)a^s. \end{aligned}$$

□

Dimostrazione del Teorema di Strassen. Dimostriamo che

$$\omega = \inf\{\tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Sia θ il lato destro dell'equazione sopra, e per semplicità scriviamo $M(n) := M_{\mathbb{K}}(n)$ e $\omega := \omega_{\mathbb{K}}$. Dall'Equazione 2.5 abbiamo

$$\frac{1}{2} \mathbf{R}(\langle n, n, n \rangle) \leq L(\langle n, n, n \rangle) \leq M(n),$$

e dalla Definizione 2.2.5 segue $\theta \leq \omega$.

Mostriamo che $\omega \leq \theta$. Per definizione di θ , per ogni $\varepsilon > 0$ esiste un $n > 1$ tale che

$$(2.11) \quad r := \mathbf{R}(\langle n, n, n \rangle) \leq n^{\theta+\varepsilon}.$$

Dall'Equazione 2.9 esistono dei funzionali $f_\rho, g_\rho \in (\mathbb{K}^{n \times n})^*$ e delle matrici $w_\rho \in \mathbb{K}^{n \times n}$ per $\rho \in \{1, \dots, r\}$ tali che, per ogni $A, B \in \mathbb{K}^{n \times n}$

$$AB = \sum_{\rho=1}^r f_\rho(A) g_\rho(B) w_\rho.$$

matrix product, ma TODO verifica, e controlla se è così davvero(io ho i vec)

Vedendo $\mathcal{A} = \mathbb{K}^{n^i \times n^i}$ come l'algebra delle matrici $n^i \times n^i$, notiamo che i funzionali $f_\rho, g_\rho : \mathbb{K}^{n \times n} \rightarrow \mathbb{K}$ si estendono a funzionali $f_\rho, g_\rho : \mathcal{A}^{n \times n} \rightarrow \mathcal{A}$, sostituendo le entrate scalari con delle matrici. In particolare vale, per ogni $A, B \in \mathcal{A}^{n \times n}$

$$(2.12) \quad AB = \sum_{\rho=1}^r f_\rho(A) g_\rho(B) w_\rho.$$

L'Equazione sopra rappresenta l'algoritmo di moltiplicazione a blocchi tra A e B : infatti abbiamo $\mathcal{A}^{n \times n} = (\mathbb{K}^{n^i \times n^i})^{n \times n} \simeq \mathbb{K}^{n^{i+1} \times n^{i+1}}$, dove l'isomorfismo è dato dalla decomposizione in blocchi. Da questo fatto, unito all'Equazione 2.12, si ottiene che

$$(2.13) \quad M(n^{i+1}) \leq rM(n^i) + cn^{2i},$$

dove $c := c(n, r)$ è una costante che dipende sia da n che da r . Questo perché nella formula il prodotto AB , sviluppato come prodotto tra blocchi $n^i \times n^i$ (formalmente, visto come prodotto a coefficienti in $\mathcal{A}$), coinvolge r prodotti e un numero costante di addizioni. Siccome $\langle n, n, n \rangle$ è concisa, abbiamo che $r \geq n^2$. Fissando n (e quindi r), risolviamo la ricorsione dell'Equazione 2.13 in funzione di i usando il Lemma 2.6.5 con $a = r$ e $b = n^2$, analizzando due casi.

Se $r > n^2$, abbiamo che

$$\begin{aligned} M(n^i) &= r^i M(1) + (r^i - n^{2i}) \frac{n^2 c}{r - n^2} \\ &= (M(1) + n^2 c) r^i - n^{2i} \frac{n^2 c}{r - n^2}, \end{aligned}$$

da cui,

$$M(n^i) \leq \alpha r^i + \beta n^{2i},$$

dove abbiamo posto $\alpha := M(1) + n^2 c$ e $\beta := -\frac{n^2 c}{r - n^2}$.

Se $r = n^2$, dal Lemma 2.13 abbiamo

$$M(n^i) \leq (cs + M(1))r^i,$$

In entrambi i casi troviamo,

$$(2.14) \quad M(n^i) = O(r^i).$$

Dalla proposizione 2.5.2 segue che M è una successione monotona, da cui per ogni $h = n^i \in \mathbb{N} \setminus \{0\}$,

$$M(h) = O(r^{\log_n h}) = O(h^{\log_n r}) = O(h^{\theta + \varepsilon}),$$

dove la prima uguaglianza segue sostituendo, $i = \log_n h$ nell'Equazione 2.13, la seconda da una proprietà del logaritmo, e la terza applicando la stima dell'Equazione 2.11. Quindi $\omega_{\mathbb{K}} \leq \theta$, che completa la dimostrazione. $\square$

2.7 ALGORITMI DI MOLTIPLICAZIONE VELOCE

Osservazione 2.7.1. Di seguito ci limiteremo a studiare prodotti fra matrici quadrate; per il caso rettangolare, si possono estendere i ragionamenti sopra considerando una rappresentazione di rango r del tensore $\langle m, n, p \rangle$ e scrivendo, attraverso una serie di permutazioni (che non specifichiamo), la fattorizzazione di $\langle mnp, mnp, mnp \rangle$ di rango r^3 , che produce un algoritmo di complessità $O(n^{\log_{abc} r^3}) = O(n^{3 \log_{abc} r})$

IL CASO NAIVE

Trattiamo il caso di matrici quadrate con taglie potenze di due (possiamo farlo a meno di passare a una sottomatrice di taglia pari). In questo caso possiamo partizionare le matrici in delle sottomatrici a blocchi 2×2 ¹:

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}.$$

L'algoritmo naive procede combinando un insieme di otto moltiplicazioni matriciali con quattro addizioni:

$$\begin{aligned} m_1 &= a_{11} \cdot b_{11} & m_2 &= a_{12} \cdot b_{21} & m_3 &= a_{11} \cdot b_{12} \\ m_4 &= a_{12} \cdot b_{22} & m_5 &= a_{21} \cdot b_{11} & m_6 &= a_{22} \cdot b_{21} \\ m_7 &= a_{21} \cdot b_{12} & m_8 &= a_{22} \cdot b_{22} \end{aligned}$$

$$\begin{aligned} c_{11} &= m_1 + m_2 = a_{11}b_{11} + a_{12}b_{21}, & c_{12} &= m_3 + m_4 = a_{11}b_{12} + a_{12}b_{22}, \\ c_{21} &= m_5 + m_6 = a_{21}b_{11} + a_{22}b_{21}, & c_{22} &= m_7 + m_8 = a_{21}b_{12} + a_{22}b_{22}. \end{aligned}$$

Usando ?? possiamo scrivere l'algoritmo $\langle 2, 2, 2 \rangle$ in forma tensoriale in formato CP:

$$\begin{aligned} \langle 2, 2, 2 \rangle &= a_{11} \circ b_{11} \circ c_{11} + a_{12} \circ b_{21} \circ c_{11} \\ &+ a_{21} \circ b_{11} \circ c_{21} + a_{22} \circ b_{21} \circ c_{21} \\ &+ a_{11} \circ b_{12} \circ c_{12} + a_{12} \circ b_{22} \circ c_{12} \\ &+ a_{21} \circ b_{12} \circ c_{22} + a_{22} \circ b_{22} \circ c_{22}. \end{aligned} \tag{2.15}$$

¹possiamo pensare alle entrate sia come scalari, che come matrici.

$$\begin{array}{c} A \\ \left[\begin{array}{|c|c|} \hline a_{11} & a_{12} \\ \hline a_{21} & a_{22} \\ \hline \end{array} \right] \end{array} \times \begin{array}{c} B \\ \left[\begin{array}{|c|c|} \hline b_{11} & b_{12} \\ \hline b_{21} & b_{22} \\ \hline \end{array} \right] \end{array} = \begin{array}{c} C \\ \left[\begin{array}{|c|c|} \hline c_{11} & c_{12} \\ \hline c_{21} & c_{22} \\ \hline \end{array} \right] \end{array}$$

$$c_{ij} = \sum_{k=1}^2 a_{ik}b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j}$$

Figura 2.2: Rappresentazione della moltiplicazione di due matrici 2×2 .

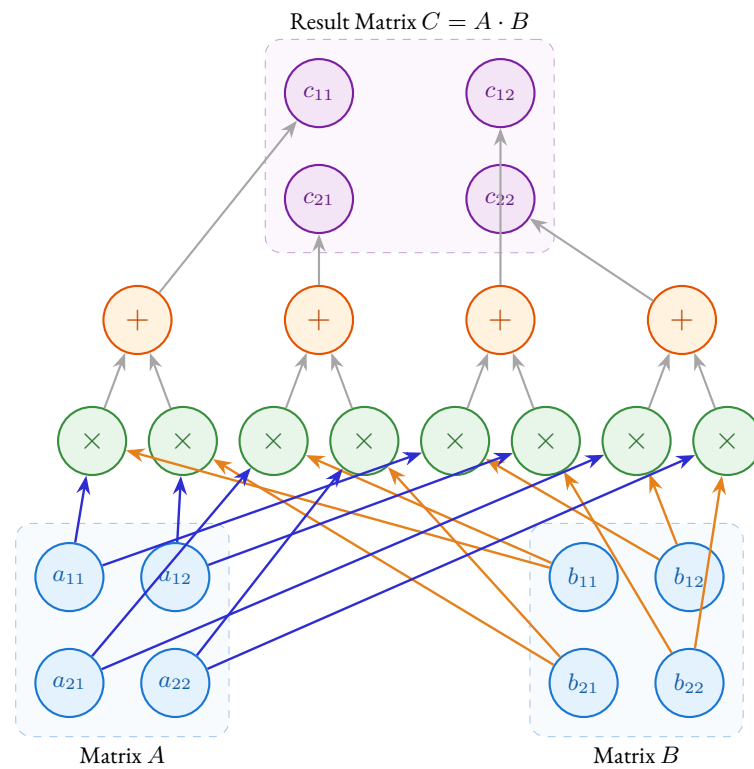


Figura 2.3: Circuito aritmetico per la moltiplicazione di matrici 2×2 .

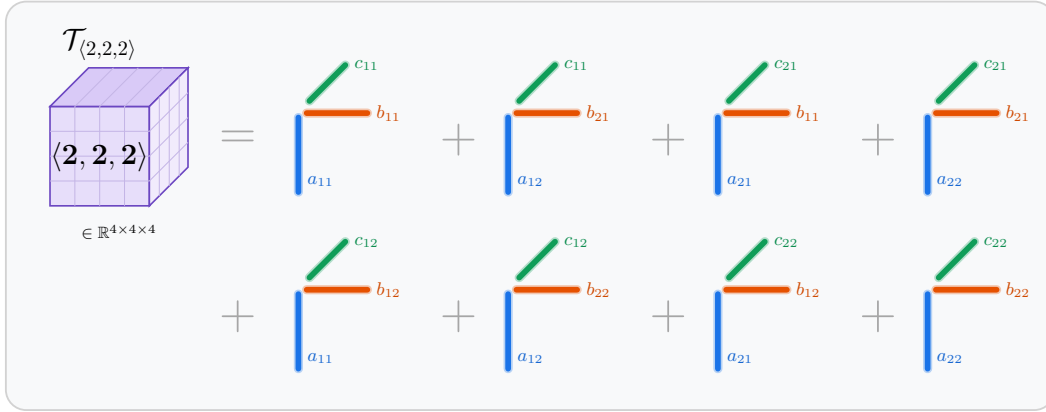


Figura 2.4: Decomposizione CP del tensore di moltiplicazione di matrici $\mathcal{T}_{(2,2,2)}$ come somma di 8 tensori di rango uno.

Per secoli questo algoritmo è stato ritenuto il più veloce per moltiplicare le matrici. Nel ?? Strassen² tentò di mostrare che non esistesse un algoritmo più veloce, fallendo in maniera spettacolare³ e dando vita a una nuova teoria.

L'ALGORITMO DI STRASSEN

L'algoritmo di Strassen è probabilmente l'algoritmo di moltiplicazione veloce più noto e segue la stessa struttura a blocchi $\langle 2, 2, 2 \rangle$.

$$\begin{array}{lll}
 s_1 = a_{11} + a_{22} & s_2 = a_{21} + a_{22} & s_3 = a_{11} \\
 s_4 = a_{22} & s_5 = a_{11} + a_{12} & s_6 = a_{21} - a_{11} \\
 s_7 = a_{12} - a_{22} & & \\
 t_1 = b_{11} + b_{22} & t_2 = b_{11} & t_3 = b_{12} - b_{22} \\
 t_4 = b_{21} - b_{11} & t_5 = b_{22} & t_6 = b_{11} + b_{12} \\
 t_7 = b_{21} + b_{22} & &
 \end{array}$$

$$\begin{array}{ll}
 m_r = s_r t_r, & 1 \leq r \leq 7 \\
 c_{11} = m_1 + m_4 - m_5 + m_7 & c_{12} = m_3 + m_5 \\
 c_{21} = m_2 + m_4 & c_{22} = m_1 - m_2 + m_3 + m_6
 \end{array}$$

²in una comunicazione riportata da Landsberg in [18].

³in realtà la tesi è vera nel campo $\mathbb{Z}/2\mathbb{Z}$.

Aggiungere ten-
sore

aggiungere co-
sto

2.8 ALGORITMI APA

Dopo il tentativo di Strassen di dimostrare che l'algoritmo di moltiplicazione standard fosse ottimale, Bini et. Al [4] considerano il seguente algoritmo di moltiplicazione ridotto ottenuto ponendo a zero l'entrata a_{22} di $\langle 2, 2, 2 \rangle$:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & 0 \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} & a_{21}b_{12} \end{bmatrix}.$$

Facendo alcuni raccoglimenti e sostituendo $a_{22} = 0$, il tensore dell'Equazione 2.15 si riscrive come

$$\begin{aligned} \langle 2, 2, 2 \rangle^{\text{red}} &:= a_{11} \circ (b_{11} \circ c_{11} + b_{12} \circ c_{12}) + a_{12} \circ (b_{21} \circ c_{11} + b_{22} \circ c_{12}) \\ &\quad + a_{21} \circ (b_{11} \circ c_{21} + b_{12} \circ c_{22}). \end{aligned}$$

Che è somma di sei tensori elementari distinti, quindi $\mathbf{R}\langle 2, 2, 2 \rangle^{\text{red}} \leq 6$. Bini et. al. provarono a cercare un'espressione di rango cinque per $\langle 2, 2, 2 \rangle^{\text{red}}$ numericamente, calcolando

$$\min_{\substack{T \in \mathbb{R}^{n \times n \times n} \\ \mathbf{R}(T)=5}} \|\langle 2, 2, 2 \rangle^{\text{red}} - T\|$$

norma di $\langle 2, 2, 2 \rangle^{\text{red}}$. Il loro computer continuava a produrre tensori T di rango cinque con la norma della differenza che diventava sempre più piccola, ma con coefficienti sempre più grandi. Questo è andato avanti per un po' di tempo, fino a che realizzarono che il codice era giusto, ma quello che sembrava un problema era in realtà una manifestazione di un fenomeno chiamato *border rank*. L'espressione del tensore che stavano cercando era il seguente tensore di rango 5:

$$\langle n, n, n \rangle_t^{\text{red}} = \frac{1}{t}[(I) + (II) - (III) - (IV) + (V)],$$

dove

$$\begin{aligned}
(I) &= (x_2^1 + tx_1^1) \otimes (y_2^1 + ty_2^2 \otimes z_2^1) \\
&= x_2^1 \otimes y_2^1 \otimes z_2^1 + t[x_2^1 \otimes y_2^2 \otimes z_2^1 + x_1^1 \otimes y_2^1 \otimes z_2^1] + O(t^2) \\
(II) &= (x_1^2 + tx_1^1) \otimes y_1^1 \otimes (z_1^1 + tz_1^2) \\
&= x_1^2 \otimes y_1^1 \otimes z_1^1 + t[x_1^2 \otimes y_1^1 \otimes z_1^2 + x_1^1 \otimes y_1^1 \otimes z_1^1] + O(t^2) \\
(III) &= x_1^2 \otimes y_1^1 \otimes z_1^1 + t[x_1^2 \otimes y_1^1 \otimes z_1^2 + x_1^1 \otimes y_1^1 \otimes z_1^1] + O(t^2) \\
(IV) &= x_1^2 \otimes ((y_1^1 + y_2^1) + ty_1^2) \otimes z_1^1 \\
&= x_1^2 \otimes y_1^1 \otimes z_1^1 + x_1^2 \otimes y_2^1 \otimes z_1^1 + t[x_1^2 \otimes y_1^2 \otimes z_1^1] \\
(V) &= (x_2^1 + x_1^2) \otimes (y_2^1 + ty_1^2) \otimes (z_1^1 + tz_2^2) \\
&= x_2^1 \otimes y_2^1 \otimes z_1^1 + x_1^2 \otimes y_2^1 \otimes z_1^1 \\
&\quad + t[x_2^1 \otimes (y_2^1 + z_2^2 + y_1^2 \otimes z_1^1) + x_1^2 \otimes (y_2^1 \otimes z_2^2 + y_1^2 \otimes z_1^1)] + O(t^2)
\end{aligned}$$

In ogni passaggio abbiamo applicato più volte la proprietà distributiva del prodotto tensoriale. Osserviamo che i termini di grado costante in t si cancellano:

$$\begin{aligned}
&x_2^1 \otimes (y_2^1 \otimes z_2^1 - y_2^1 \otimes z_1^1 - y_2^1 \otimes z_2^2 + y_2^1 \otimes z_1^1) \\
&+ x_1^2 \otimes (y_1^1 \otimes z_1^1 - y_1^1 \otimes z_1^1 + y_1^1 \otimes z_1^1 - y_1^1 \otimes z_1^1) = 0
\end{aligned}$$

Sommando e raggruppando i termini di grado uno in t troviamo

$$\begin{aligned}
\langle n, n, n \rangle^{\text{red}} &= x_1^1 \otimes (y_1^1 \otimes z_1^1 + y_2^1 \otimes z_2^1) + \\
&\quad x_2^1 \otimes (y_1^2 \otimes z_1^1 + y_2^2 \otimes z_2^1) + \\
&\quad x_1^2 \otimes (y_1^1 \otimes z_1^2 + y_2^1 \otimes z_2^2),
\end{aligned}$$

ovvero il tensore di partenza. Da cui,

$$\langle n, n, n \rangle_t^{\text{red}} - \langle n, n, n \rangle^{\text{red}} = O(t^2) \rightarrow 0$$

Abbiamo dimostrato che un tensore di rango 5 converge a uno di rango 6.

IL PROBLEMA DEL BORDER RANK

Esempio 2.8.1. Consideriamo dei vettori $a_1, a_2 \in \mathbb{R}^m$, $b_1, b_2 \in \mathbb{R}^n$ e $c_1, c_2 \in \mathbb{R}^p$ vettori linearmente indipendenti. Definiamo il tensore

$$(2.16) \quad T = a_1 \circ b_1 \circ c_2 + a_1 \circ b_2 \circ c_1 + a_2 \circ b_1 \circ c_1,$$

Introduci e motiva il problema partendo dagli algoritmi APA (vedi Landsbeg)

e notiamo che $\mathbf{R}(T) = 3$ dato che non può essere scomposto come somma di tensori più semplici. Consideriamo la successioni di tensori

$$(2.17) \quad T_\varepsilon = \frac{1}{\varepsilon}(a_1 + \varepsilon a_2) \circ (b_1 + \varepsilon b_2) \circ (c_1 + \varepsilon c_2) - \frac{1}{\varepsilon}a_1 \circ b_1 \circ c_1, \text{ con } \varepsilon \in \mathbb{R},$$

che essendo combinazione lineare di due tensori di rango uno, ha rango 2. Vediamo che $\lim_{\varepsilon \rightarrow 0} T_\varepsilon = T$, da cui segue che l'insieme dei tensori di rango 3 non è chiuso. Espandendo i termini dell'Equazione 2.17 tramite la proprietà distributiva del prodotto esterno,

$$(2.18) \quad \begin{aligned} \mathbf{Y}_\varepsilon &= \frac{1}{\varepsilon}a_1 \circ b_1 \circ c_2 + a_1 \circ b_1 \circ c_2 + a_1 \circ b_2 \circ c_1 + a_2 \circ b_1 \circ c_1 \\ &\quad + \varepsilon(a_2 \circ b_2 \circ c_1 + a_1 \circ b_1 \circ c_2 + a_1 \circ b_2 \circ c_2 + a_2 \circ b_2 \circ c_2) - \frac{1}{\varepsilon}a_1 \circ b_1 \circ c_1 \\ &= T + \varepsilon(a_2 \circ b_2 \circ c_1 + a_1 \circ b_1 \circ c_2 + a_1 \circ b_2 \circ c_2 + a_2 \circ b_2 \circ c_2) \end{aligned}$$

Quindi $T_\varepsilon - T \rightarrow 0$.

Questo motiva la seguente definizione:

Definizione 2.8.2 (Border rank). Un tensore T ha *border rank* r se è limite di una successione di tensori di rango r , ma non è limite di una successione di tesori di rango s , per ogni $s < r$. Lo denoteremo con il simbolo $\underline{\mathbf{R}}(T)$.

Osservazione 2.8.3. $\mathbf{R}(T) \geq \underline{\mathbf{R}}(T)$.

Osservazione 2.8.4. In coordinate, se $T \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$, il suo border rank è la quantità

$$\underline{\mathbf{R}}(T) = \min\{r \in \mathbb{N} \mid \exists T' \in \mathbb{R}^{n_1 \times \dots \times n_d} \text{ tale che } \mathbf{R}(T') = r, \|T - T'\| < \varepsilon\}.$$

Disegno interpretazione geometrica (landsberg pg 38)

Il border rank si rivela utilissimo nello studio dell'esponente ω , cioè possiamo sostituire il rango con il border rank nel Teorema 2.6.1:

Teorema 2.8.5 (Bini [4], vedi §3.2).

$$\omega = \inf\{\tau \in \mathbb{R} \mid \underline{\mathbf{R}}(\langle n, n, n \rangle) = O(n^\tau)\}.$$

Anche se potremmo avere $\underline{\mathbf{R}}(\langle n, n, n \rangle) < \mathbf{R}(\langle n, n, n \rangle)$, non sono abbastanza diversi da avere un impatto sull'esponente.

why?

Riportiamo alcune stime sul border rank riportate in [17], §1. Per $n = 2$, Winograd dimostra che $\underline{\mathbf{R}}(\langle 2, 2, 2 \rangle) = \mathbf{R}(\langle 2, 2, 2 \rangle) = 7$. Ci aspetteremmo che per $n > 2$,

ref

$\underline{\mathbf{R}}(\langle n, n, n \rangle) < \mathbf{R}(\langle n, n, n \rangle)$. Per $n = 3$, sappiamo solo che $15 \leq \underline{\mathbf{R}}(\langle 3, 3, 3 \rangle) \leq 20$ e $19 \leq \mathbf{R}(\langle 3, 3, 3 \rangle) \leq 23$. In generale, sappiamo che $\mathbf{R}(\langle n, n, n \rangle) \geq 3n^2 - o(n)$ e $\underline{\mathbf{R}}(\langle n, n, n \rangle) \geq 2n^2 - \lceil \log_2(n) \rceil - 1$.

2.9 ERRORE ALGORITMICO

discussione della stabilità dal paper dato dal prof

3

APPLICAZIONI DELLA CP

cambia titolo

Le decomposizioni di tensori occorrono in numerose aree scientifiche. Alcune di queste sono:

1. Nella psicomетria, dove è stata impiegata per valutare l'intelligenza e alcune caratteristiche personali. Alcuni dei primi lavori in quest'area includono [311, 71, 158].
 2. Nella geofisica; ad esempio l'interpretazione di data magnetotellurica per strutture regionali a una e due dimensioni, ad esempio in ...e le reference all'interno.
 3. Nell'interpretare la immagini di risonanze magnetiche (MRI); vedi ad esempio e relative reference. Un utilizzo pratico è trovare l'area del cervello in cui è in atto una crisi epilettica in un paziente. Un altro impeiego è la diagnosi e
 4. Nella data mininig
 5. Nell'analisi numerica. Grazie alla convergenza delle serie di Fourier in $L^2(\mathbb{R}^n)$, si ha $L^2(\mathbb{R}^n) = L^2(\mathbb{R}) \otimes \dots \otimes L^2(\mathbb{R})$, e spesso vengono approssimate funzioni di n variabili da una somma finita di prodotti di funzioni in una variabile, generalizzando la separazione di variabili. Si veda ...
- In geophysics; for example the interpretation of magnetotelluric data for one-dimensional and two-dimensional regional structures; see, e.g., [335] and the references therein.
 - In interpreting magnetic resonance imaging (MRI); see, e.g., [284, 125], and the references therein. One such use is in determining the location in the brain that is causing epileptic seizures in a patient. Another is the diagnosis and management of strokes.

3.1 FLUORESCENZA SPECTROSCOPICA

La fluorescenza spettroscopica è una tecnica della chimica che ...

Principali references: [22]§10.2, for descriptions: [3, 18]

Vengono dati I campioni di soluzioni chimiche, ognuno dei quali contiene diverse sostanze diluite in diverse concentrazioni. Il primo obiettivo è determinare il numero r delle diverse sostanze presenti. In ci sono quattro di queste, *dihydroxybenzene*, *tryptophan*, *phenylalanine* e DOPA. Ogni campione, diciamo il numero i -esimo è successivamente eccitato da una luce in J diverse lunghezze d'onda. Per ogni lunghezza d'onda di eccitazione, viene misurato lo spettro emesso. Diciamo che l'intensità della luce fluorescente emessa è misurata da K diverse lunghezze d'onda. Allora per ogni i , otteniamo una tabella $J \times K$ di matrici di eccitazione-emissione.

Quindi i dati possono essere rappresentati come un array $I \times J \times K$. In basi, se $\{e_i\}$ è la base di $\mathbb{C}^I$, h_j è una base di $\mathbb{C}^J$, e g_k è una base di $\mathbb{C}^K$, allora $T = \sum_{ijk} T_{ijk} e_i \otimes h_j \otimes g_k$. Il primo obiettivo è trovare un r tale che

$$T \simeq \sum_{f=1}^r a_f \otimes b_f \otimes c_f,$$

dove ogni f rappresenta una sostanza. Scrivendo $a_f = \sum a_{i,f} e_i$, allora $a_{i,f}$ è la concentrazione della f -esima sostanza nell' i -esimo campione, e in maniera simile usando le basi di $\mathbb{R}^J$ e $\mathbb{R}^K$, $c_{k,f}$ è la frazione di protoni che la f -esima sostanza emette nella lunghezza d'onda k , e $b_{j,f}$ è l'intensità della luce incidente alla lunghezza d'onda j moltiplicata per l'assorbimento alla lunghezza d'onda j .

Siccome i dati conterranno rumore, possiamo assumere T di rango generico, ma esisterà un tensore di rango molto basso $\tilde{T}$ che lo approssima fedelmente (). Gli autori di [22] non associano alcuna metrica ai dati, quindi il significato di "approssimazione" non è chiaro. In [22] si procede come segue per trovare r . Prima di tutto, si assume che r sia molto basso (al più 7 nella loro esposizione). Poi per ogni r_0 tale che $1 \leq r_0 \leq 7$, si assume $r_0 = r$ e si applica un algoritmo numerico che tenta di trovare le r_0 componenti (i.e, tensori di rango uno) che formano $\tilde{T}$. I valori di r_0 per cui l'algoritmo non converge velocemente sono scartati (gli autori sottolineano che questa procedura non è giustificata matematicamente, ma sembra funzionare bene in pratica; nell'esempio, questi valori di r_0 sono troppo grandi). Poi, per i restanti valori di r_0 , vengono analizzati i tensori risultati per vedere se sono ragionevoli dal punto di vista fisico. Questo rende loro possibile di rimuovere valori di r_0 troppo piccoli. Nell'esempio, a loro restano $r_0 = 4, 5$.

Assumiamo ora che r sia stato determinato. Siccome i valori di r sono relativamente piccoli, a meno di banalità, l'espressione di $\tilde{T}$ come somma di elementi di rango uno sarà unica. Quindi, decomponendo $\tilde{T}$, possiamo recuperare la concen-

ref

qui aggiustare
con i dataset di
Ballard

ref to generic
rank

why?

funziona kru-
skal?

tazione di ognuna delle r sostanze in ogni soluzione, determinando i vettori a_f così come lo spettro delle eccitazioni ed emissioni determinando i vettori b_f .

3.2 COCKTAIL PARTY PROBLEM

Un problema centrale nel signal processing è il *problema del cocktail party*, dove un insieme di persone si trova in una stanza a parlare. Ci sono diversi ricevitori impostati che registrano le conversazioni: non singolarmente ma tutte insieme, sovrapposte. L'obiettivo è recuperare cosa ogni persona sta dicendo. Questo "unmixing" si può ottenere usando la decomposizione CP.

Explain

4 APPENDICE

4.1 IL MASTER METHOD

Introduciamo brevemente un metodo per risolvere relazioni ricorsive definite per ricorrenza, detto *master method*. I lettori più interessati possono trovare una trattazione più dettagliata sul libro [9]. Il master method si usa per risolvere ricorrenze della forma

$$(4.1) \quad T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

dove $a \geq 1$ e $b > 1$ sono costanti e $f(n)$ è una funzione asintoticamente positiva. La ricorrenza 4.1 descrive il tempo di esecuzione di un algoritmo che divide un problema di dimensione n in a sottoproblemi, ognuno di dimensione $\frac{n}{b}$. Gli a sottoproblemi vengono risolti ricorsivamente in tempo $T(\frac{n}{b})$, e la funzione $f(n)$ rappresenta il costo di dividere il problema e ricombinare la ricorsione di ogni sottoproblema.

Definizione 4.1.1 (Notazione O –grande/piccolo). Per delle funzioni f, g a variabile reale (o intera) nella variabile x diciamo che:

- $f(x) = O(g(x))$ se esiste una costante $C > 0$ e un valore x_0 tale che $|f(x)| \leq C|g(x)|$ per ogni $x \geq x_0$.
- $f(x) = o(g(x))$ se $\lim_{x \rightarrow \infty} \frac{|f(x)|}{|g(x)|} = 0$.
- $f(x) = \Omega(g(x))$ se esiste una costante $C > 0$ e un valore x_0 tale che $C|f(x)| \geq |g(x)|$ per ogni $x \geq x_0$.
- $f(x) = \omega(g(x))$ se $\lim_{x \rightarrow \infty} \frac{|g(x)|}{|f(x)|} = 0$.
- $f(x) = \Theta(g(x))$ se $f(x) = O(g(x))$ e $f(x) = \Omega(g(x))$.

Teorema 4.1.2 (Master theorem). Siano $a \geq 1$ e $b > 1$ delle costanti, sia $f(n)$ una funzione e definiamo la legge ricorsiva $T : \mathbb{N} \rightarrow \mathbb{R}$ con la ricorrenza

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

dove con $\frac{n}{b}$ indichiamo sia $\lfloor \frac{n}{b} \rfloor$ o $\lceil \frac{n}{b} \rceil$. Allora $T(n)$ assume i seguenti comportamenti asintotici:

1. Se $f(n) = O(n^{\log_b a - \varepsilon})$ per qualche $\varepsilon > 0$, allora $T(n) = \Theta(n^{\log_b a})$.
2. Se $f(n) = \Theta(n^{\log_b a})$ allora $T(n) = \Theta(n^{\log_b a \log n})$.
3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ per qualche costante $\varepsilon > 0$, e se $af(\frac{n}{b}) < cf(n)$ per qualche costante $c < 1$ e ogni n sufficientemente grande, allora $T(n) = \Theta(f(n))$.

Esempio 4.1.3. Nel caso dell'algoritmo naive la ricorrenza ha la forma $a = 8$, $b = 2$ e $f(n) = \Theta(n^2)$, ovvero

$$T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n^2).$$

Siccome $n^{\log_b a} = n^{\log_2 8} = n^3$, che è polinomialmente più grande di $f(n)$, ovvero $f(n) = O(n^{3-\varepsilon})$ per $\varepsilon = 1$, dal caso 1 di 4.1.2 segue che $T(n) = \Theta(n^3)$. Nel caso di Strassen, si ripetono i calcoli con $a = 7$, $b = 2$ e $f(n) = \Theta(n^2)$ e si ottiene la relazione $T(n) = 7T(\frac{n}{2}) + \Theta(n^2)$ e analogamente $T(n) = \Theta(n^{\log_2 7})$.

ACRONIMI

PCA	Principal component analysis
SNF	Smith normal form
TDA	Topological data analysis

GLOSSARIO

$\text{\LaTeX}$	A document preparation system
$\mathbb{R}$	The set of real numbers

BIBLIOGRAFIA

1. J. Alman, R. Duan, V. V. Williams, Y. Xu, Z. Xu, e R. Zhou. «More asymmetry yields faster matrix multiplication». *Computing Research Repository (arXiv)*, 2024. arXiv:2404.16349.
2. J. Alman, V. Vassilevska Williams et al. «Improving the matrix multiplication exponent with modern optimization and AlphaEvolve». *arXiv preprint*, 2026.
3. G. Ballard e T. G. Kolda. *Tensor Decompositions for Data Science*. Cambridge University Press, 2025. URL: <https://tensortextbook.com>.
4. D. Bini. «Relations between exact and approximate bilinear algorithms. Applications». *Calcolo* 17:1, 1980, pp. 87–97. DOI: [10.1007/bf02575865](https://doi.org/10.1007/bf02575865). URL: <https://doi.org/10.1007/bf02575865>.
5. M. Bläser. «A $(5/2)n^2$ -Lower Bound for the Multiplicative Complexity of $n \times n$ -Matrix Multiplication». In: *Proceedings of the 18th Annual Symposium on Theoretical Aspects of Computer Science*. STACS '01. Springer-Verlag, Berlin, Heidelberg, 2001, pp. 99–109. ISBN: 3540416951.
6. M. Bläser. «On the complexity of the multiplication of matrices of small formats». *Journal of Complexity* 19:1, 2003, pp. 43–60. ISSN: 0885-064X. DOI: [https://doi.org/10.1016/S0885-064X\(02\)00007-9](https://doi.org/10.1016/S0885-064X(02)00007-9). URL: <https://www.sciencedirect.com/science/article/pii/S0885064X02000079>.
7. P. Bürgisser, M. Clausen, e M. A. Shokrollahi. *Algebraic complexity theory*. Vol. 315. Springer Science & Business Media, 2013.
8. D. Coppersmith e S. Winograd. «Matrix multiplication via arithmetic progressions». *Journal of Symbolic Computation* 9:3, 1990. Computational algebraic complexity editorial, pp. 251–280. ISSN: 0747-7171. DOI: [https://doi.org/10.1016/S0747-7171\(08\)80013-2](https://doi.org/10.1016/S0747-7171(08)80013-2). URL: <https://www.sciencedirect.com/science/article/pii/S0747717108800132>.
9. T. H. Cormen, C. E. Leiserson, R. L. Rivest, e C. Stein. *Introduction to algorithms*. MIT press, 2022.

10. P. D'Alberto. *Strassen's Matrix Multiplication Algorithm Is Still Faster*. 2023. arXiv: [2312.12732](https://arxiv.org/abs/2312.12732) [cs.MS]. URL: <https://arxiv.org/abs/2312.12732>.
11. L. DeLathauwer. «A Link between the Canonical Decomposition in Multilinear Algebra and Simultaneous Matrix Diagonalization». *SIAM Journal on Matrix Analysis and Applications* 28:3, 2006, pp. 642–666. DOI: [10.1137/040608830](https://doi.org/10.1137/040608830). eprint: <https://doi.org/10.1137/040608830>. URL: <https://doi.org/10.1137/040608830>.
12. C. Eckart e G. Young. «The approximation of one matrix by another of lower rank». *Psychometrika* 1:3, 1936, pp. 211–218.
13. A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatin, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz et al. «Discovering faster matrix multiplication algorithms with reinforcement learning». *Nature* 610:7930, 2022, pp. 47–53.
14. F. L. Gall. «Powers of Tensors and Fast Matrix Multiplication». *CoRR* abs/1401.7714, 2014. arXiv: [1401.7714](http://arxiv.org/abs/1401.7714). URL: <http://arxiv.org/abs/1401.7714>.
15. J. Håstad. «Tensor rank is NP-complete». In: *International colloquium on automata, languages, and programming*. Springer. 1989, pp. 451–460.
16. J. D. Laderman. «A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications». *Bulletin of the American Mathematical Society* 82:1, 1976, pp. 126 –128.
17. J. M. Landsberg. «Geometry and complexity theory». *NSF Award Number 1405348. Directorate for Mathematical and Physical Sciences* 14:1405348, 2015, p. 5348.
18. J. M. Landsberg. *Tensors: geometry and applications*. Vol. 128. American Mathematical Soc., 2011.
19. V. Y. Pan. «Strassen's algorithm is not optimal trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations». In: *19th Annual Symposium on Foundations of Computer Science (sfcs 1978)*. 1978, pp. 166–176. DOI: [10.1109/SFCS.1978.34](https://doi.org/10.1109/SFCS.1978.34).
20. A. Schönhage. «Partial and Total Matrix Multiplication». *SIAM Journal on Computing* 10:3, 1981, pp. 434–455. DOI: [10.1137/0210032](https://doi.org/10.1137/0210032). eprint: <https://doi.org/10.1137/0210032>. URL: <https://doi.org/10.1137/0210032>.

21. V. de Silva e L.-H. Lim. *Tensor rank and the ill-posedness of the best low-rank approximation problem*. 2008. arXiv: [math/0607647](https://arxiv.org/abs/math/0607647) [math.NA]. URL: <https://arxiv.org/abs/math/0607647>.
22. A. K. Smilde, R. Bro, P. Geladi et al. *Multi-way analysis with applications in the chemical sciences*. Vol. 396. Wiley Chichester, 2004.
23. A. Stothers. «On the complexity of matrix multiplication». Tesi di dott. Edinburgh, UK: University of Edinburgh, 2010. URL: <https://ed.ac.uk>.
24. V. STRASSEN. «Relative bilinear complexity and matrix multiplication.» *Journal für die reine und angewandte Mathematik* 1987:375-376, 1987, pp. 406–443. DOI: [doi:10.1515/crll.1987.375-376.406](https://doi.org/10.1515/crll.1987.375-376.406). URL: <https://doi.org/10.1515/crll.1987.375-376.406>.
25. V. Strassen. «Gaussian elimination is not optimal». *Numer. Math.* 13:4, 1969, pp. 354–356. ISSN: 0029-599X. DOI: [10.1007/BF02165411](https://doi.org/10.1007/BF02165411). URL: <https://doi.org/10.1007/BF02165411>.
26. V. V. Williams. *Breaking the Coppersmith-Winograd barrier*. Manuscript available at <http://theory.stanford.edu/~virgi/matrixmult-f.pdf>. 2011.
27. S. Winograd. «On multiplication of 2×2 matrices». *Linear Algebra and its Applications* 4:4, 1971, pp. 381–388. ISSN: 0024-3795. DOI: [https://doi.org/10.1016/0024-3795\(71\)90009-7](https://doi.org/10.1016/0024-3795(71)90009-7). URL: <https://www.sciencedirect.com/science/article/pii/0024379571900097>.